Polynomials Trajectory Planner

Preliminary Documentation

Gianluca Castellari

Summer 2026

Abstract

Symbology, nomenclature definitions. Theory of the polynomial profiles.

Contents

1 Vision **7**
- 1.1 Outputs . . . 8
- 1.2 Topic classification . . . 8
- 1.3 General rules (apply throughout, unless a requirement says otherwise) . . . 8

2 Requirements **8**

3 Data Model **9**
- 3.1 Points . . . 9
- 3.2 Segments and Trajectories . . . 10
- 3.3 Polynomial and boundary conditions . . . 10
- 3.4 Constraints and Blend . . . 10
- 3.5 State (conceptual view) . . . 11
- 3.6 How the entities relate . . . 11

4 Assumptions **11**

5 Future Topics **11**
- 5.1 Orientation . . . 11
- 5.2 Time scaling . . . 12
- 5.3 Multi-axis synchronization . . . 12
- 5.4 Path vs Trajectory . . . 12
- 5.5 Numerical Stability . . . 12
- 5.6 Implementation notes carried over from the original brief . . . 12

6 Success Criteria **12**

7 Theory Library Roadmap **12**
- 7.1 Format convention . . . 13
- 7.2 Pre-publication verification checklist . . . 13
- 7.3 Out of scope for this roadmap (tracked elsewhere) . . . 14

8 Implementation Plan **14**
- 8.1 Intro And Scope . . . 14
- 8.2 Consolidated Decisions . . . 15
  - 8.2.1 Decision 1 — Segment/Blend/Transition Duration Strategy . . . 15
  - 8.2.2 Decision 2 — Boundary Condition / Waypoint State Representation . . . 15
  - 8.2.3 Decision 3 — Absolute Vs Percentage Specification . . . 15
  - 8.2.4 Decision 4 — Numerical Strategy For Non-Closed-Form Cases . . . 16
  - 8.2.5 Decision 5 — Smoothness Criterion For Exact Pass-Through . . . 16
  - 8.2.6 Decision 6 — Polynomial Evaluation Form . . . 16

8.3 Deferred Decisions . . . 16
8.3.1 Deferred — UI Exposure Choices . . . 16
8.3.2 Deferred — Charting Library . . . 16
8.4 Open Points Carried Forward . . . 17
8.5 Project Structure . . . 17
8.6 Suggested Milestone Sequence . . . 17
8.7 Relation To The Data Model . . . 17
8.8 Appendix — Stable Facts Emerged During Initial Implementation . . . 17
8.8.1 A1 — Decision 2 Was Refined Into Typed 1D/3D Boundary Conditions . . . 17
8.8.2 A2 — The Duration Abstraction Was Concretized . . . 18
8.8.3 A3 — Trajectory Construction Uses Builder-Style Services . . . 18
8.8.4 A4 — The Test Stack Is xUnit And Remains Core-Only . . . 18
8.8.5 A5 — Milestone M2 Evolved Into A Chart-First Prototype Layout . . . 18
8.9 Related Documents . . . 19

9 UI First Prototype **19**
9.1 Intro And Scope . . . 19
9.2 Operational Flow . . . 19
9.3 Input Requirements . . . 19
9.4 Output Requirements . . . 20
9.5 Explicitly Out Of Scope For Now . . . 20
9.6 Resolution Of Open Points From 07-implementation-plan.md . . . 20
9.7 Charting Library . . . 20
9.8 Implemented Clarifications . . . 21
9.9 Relation To 07-implementation-plan.md . . . 21
9.10 Related Documents . . . 21

10 Symbols And Nomenclature **21**
10.1 Intro And Scope . . . 21
10.2 Guiding Conventions . . . 21
10.3 Time Variables . . . 21
10.4 Position And Trajectory In 1D . . . 22
10.5 Derivatives In 1D . . . 22
10.6 Boundary Conditions In 1D . . . 23
10.7 State Notation . . . 23
10.8 Position And Trajectory In 3D . . . 23
10.9 Derivatives In 3D . . . 24
10.10 Boundary Conditions In 3D . . . 24
10.11 Segments, Multi-Segment Trajectories, And Blending . . . 24
10.12 Constraint Notation . . . 25
10.13 Reserved Orientation Notation . . . 25
10.14 Procedure For Later Documents . . . 26
10.15 Worked Symbolic Considerations . . . 26
10.16 Open Questions . . . 26
10.17 Related Documents . . . 26

11 Ninth-Degree Polynomial Theory **27**
11.1 Intro And Scope . . . 27
11.2 Symbols Used . . . 27
11.3 Why Degree Nine Is A Natural First Case . . . 27
11.4 Canonical Form In The Time Domain . . . 27
11.4.1 Reading The Canonical Form . . . 28
11.5 Horner Form In The Time Domain . . . 28
11.5.1 How Horner Form Is Built . . . 28
11.6 Canonical Form Vs Horner Form . . . 28
11.6.1 Canonical Form: Advantages . . . 29
11.6.2 Canonical Form: Limitations . . . 29
11.6.3 Horner Form: Advantages . . . 29
11.6.4 Horner Form: Limitations . . . 29

11.6.5 Practical Interpretation . . . 29
11.7 General Derivative Rule . . . 29
11.8 Derivatives Of The Ninth-Degree Polynomial . . . 30
11.8.1 First Derivative: Velocity . . . 30
11.8.2 Second Derivative: Acceleration . . . 30
11.8.3 Third Derivative: Jerk . . . 30
11.8.4 Fourth Derivative: Snap . . . 30
11.8.5 Fifth Derivative: Crackle . . . 30
11.8.6 Sixth Derivative: Pop . . . 30
11.9 What Changes In 3D . . . 30
11.10 Why Fewer Derivatives May Be Enough . . . 31
11.10.1 Position And Velocity Only . . . 31
11.10.2 Up To Acceleration . . . 31
11.10.3 Up To Jerk . . . 31
11.11 Why More Derivatives Can Be Useful . . . 31
11.11.1 First Reason: Endpoint Specification Richness . . . 31
11.11.2 Second Reason: Higher-Order Smoothness . . . 31
11.11.3 Third Reason: Analytical Support For Later Constraints . . . 32
11.12 Are Crackle And Pop Physically Necessary . . . 32
11.13 The Special Role Of Snap In Degree Nine . . . 32
11.14 A Practical Evaluation Procedure . . . 32
11.15 Worked Symbolic Considerations . . . 32
11.15.1 Example 1: Full Ninth-Degree Segment In 1D . . . 32
11.15.2 Example 2: Why Higher Derivatives Matter For Extrema . . . 33
11.15.3 Example 3: 3D Interpretation . . . 33
11.16 Open Questions . . . 33
11.17 Related Documents . . . 33

12 Boundary Conditions **33**
12.1 Intro And Scope . . . 33
12.2 Symbols Used . . . 34
12.3 What A Boundary Condition Is . . . 34
12.4 Why The Number Of Boundary Conditions Depends On Degree . . . 34
12.5 The Natural Symmetric Endpoint Pattern . . . 35
12.6 Three Condition Classes . . . 35
12.6.1 Necessary Conditions . . . 35
12.6.2 Possible Conditions . . . 35
12.6.3 Redundant Conditions . . . 36
12.7 Condition Catalogue In 1D . . . 36
12.8 Degree 3 . . . 36
12.8.1 Polynomial Structure . . . 36
12.8.2 Necessary Conditions . . . 36
12.8.3 Possible Conditions . . . 37
12.8.4 Redundant Conditions . . . 37
12.9 Degree 5 . . . 37
12.9.1 Polynomial Structure . . . 37
12.9.2 Necessary Conditions . . . 37
12.9.3 Possible Conditions . . . 37
12.9.4 Redundant Conditions . . . 38
12.10 Degree 7 . . . 38
12.10.1 Polynomial Structure . . . 38
12.10.2 Necessary Conditions . . . 38
12.10.3 Possible Conditions . . . 38
12.10.4 Redundant Conditions . . . 38
12.11 Degree 9 . . . 38
12.11.1 Polynomial Structure . . . 38
12.11.2 Necessary Conditions . . . 39
12.11.3 Possible Conditions . . . 39
12.11.4 Redundant Conditions . . . 39

12.12 Summary Table For 1D . . . 39
12.13 Independence Matters As Much As Count . . . 40
12.14 What Changes In 3D . . . 40
12.15 Relation To The Data Model . . . 40
12.16 Worked Symbolic Considerations . . . 41
12.16.1 Example 1: Cubic Segment . . . 41
12.16.2 Example 2: Quintic Segment . . . 41
12.16.3 Example 3: Ninth-Degree Segment . . . 41
12.17 Open Questions . . . 41
12.18 Related Documents . . . 41

13 Polynomial Coefficient Determination **42**
13.1 Intro And Scope . . . 42
13.2 Symbols Used . . . 42
13.3 Problem Statement . . . 42
13.4 Core Idea . . . 43
13.5 Absolute-Time Formulation . . . 43
13.5.1 General Polynomial . . . 43
13.5.2 General Derivative Rule . . . 43
13.5.3 Derivative Row Operators . . . 43
13.6 Degree-Specific Linear Systems In Absolute Time . . . 44
13.6.1 Degree 3 . . . 44
13.6.2 Degree 5 . . . 44
13.6.3 Degree 7 . . . 45
13.6.4 Degree 9 . . . 45
13.7 Step-By-Step Procedure In Absolute Time . . . 45
13.8 Why Normalized Time Is Worth Introducing . . . 46
13.9 Derivative Scaling Between t And τ . . . 46
13.10 Degree-9 System In Normalized Time . . . 47
13.11 How The Lower Degrees Fit The Same Normalized Pattern . . . 48
13.11.1 Degree 3 . . . 48
13.11.2 Degree 5 . . . 48
13.11.3 Degree 7 . . . 48
13.11.4 Degree 9 . . . 48
13.12 Absolute Time Vs Normalized Time . . . 48
13.12.1 Absolute Time: Main Advantages . . . 49
13.12.2 Absolute Time: Main Limitations . . . 49
13.12.3 Normalized Time: Main Advantages . . . 49
13.12.4 Normalized Time: Main Limitations . . . 49
13.12.5 Practical Interpretation For This Project . . . 49
13.13 Recommended Determination Strategy . . . 49
13.14 Independence And Solvability . . . 49
13.15 Relation To The Data Model . . . 50
13.16 What Changes In 3D . . . 50
13.17 Worked Symbolic Considerations . . . 51
13.17.1 Example 1: Degree 3 . . . 51
13.17.2 Example 2: Degree 9 In Normalized Time . . . 51
13.17.3 Example 3: Why Normalized Time Helps In Multi-Segment Work . . . 51
13.18 Open Questions . . . 51
13.19 Related Documents . . . 51

14 Trajectory Constraints **52**
14.1 Intro And Scope . . . 52
14.2 Symbols Used . . . 52
14.3 Constraint Context . . . 52
14.4 Core Principle: Candidate Points For Extrema . . . 53
14.5 Why Higher Derivatives Matter For Extrema . . . 53
14.6 Position Extrema In 1D . . . 53
14.6.1 General Rule . . . 53

14.6.2 Degree-9 Specialization . . . 53
14.6.3 Important Interpretation . . . 54
14.7 Velocity Extrema In 1D . . . 54
14.7.1 General Rule . . . 54
14.7.2 Degree-9 Specialization . . . 54
14.7.3 Absolute Velocity . . . 54
14.8 Acceleration Extrema In 1D . . . 54
14.8.1 General Rule . . . 54
14.8.2 Degree-9 Specialization . . . 55
14.8.3 Absolute Acceleration . . . 55
14.9 Jerk Extrema In 1D . . . 55
14.9.1 General Rule . . . 55
14.9.2 Degree-9 Specialization . . . 55
14.9.3 Absolute Jerk . . . 55
14.10 Snap Extrema In 1D . . . 56
14.10.1 General Rule . . . 56
14.10.2 Degree-9 Specialization . . . 56
14.11 Summary Of Stationarity Equations . . . 56
14.12 Symbolic Setup Vs Numerical Root Extraction . . . 56
14.13 Generic 1D Extrema Algorithm . . . 57
14.14 Practical Root-Finding Note . . . 57
14.15 Imposing A Velocity Limit In 1D . . . 57
14.15.1 Problem Form . . . 57
14.15.2 Verification Step . . . 57
14.15.3 Constraint-Imposition Strategy . . . 58
14.15.4 Duration Search . . . 58
14.16 Imposing An Acceleration Limit In 1D . . . 58
14.16.1 Problem Form . . . 58
14.16.2 Verification Step . . . 58
14.16.3 Constraint-Imposition Strategy . . . 58
14.17 Imposing A Jerk Limit In 1D . . . 59
14.17.1 Problem Form . . . 59
14.17.2 Verification Step . . . 59
14.17.3 Constraint-Imposition Strategy . . . 59
14.18 A Caution About Time Scaling . . . 59
14.18.1 Special Normalized-Profile Case . . . 60
14.19 Position, Velocity, Acceleration, Jerk, And Snap In Normalized Time . . . 60
14.20 What Changes In 3D . . . 60
14.20.1 Component-Wise Analysis . . . 60
14.20.2 Vector-Magnitude Analysis . . . 61
14.21 Recommended 3D Interpretation For This Project . . . 61
14.22 Relation To The Data Model . . . 61
14.23 Generic Constraint-Verification Algorithm . . . 62
14.24 Generic Constraint-Imposition Algorithm . . . 62
14.25 Worked Symbolic Considerations . . . 62
14.25.1 Example 1: Position Extrema . . . 62
14.25.2 Example 2: Velocity Limit . . . 62
14.25.3 Example 3: 3D Constraint Interpretation . . . 63
14.26 Open Questions . . . 63
14.27 Related Documents . . . 63

15 Constant Velocity Segment **63**
15.1 Intro And Scope . . . 63
15.2 Symbols Used . . . 64
15.3 Path Vs Trajectory: Why This Topic Is Special . . . 64
15.4 First Key Fact: Exact Constant Velocity Is Very Restrictive . . . 65
15.5 1D Case: Exact Constant Velocity Segment . . . 65
15.5.1 Geometric Interpretation . . . 65
15.6 Duration Of The Constant-Velocity Part . . . 66

15.7 Why A Multi-Segment Reformulation Is Needed . . . 66
15.8 Entry And Exit States In 1D . . . 66
15.9 Baseline 1D Construction Strategy . . . 66
15.10 How The Start And End Positions May Be Specified In 1D . . . 67
15.10.1 Absolute Specification . . . 67
15.10.2 Percentage Specification . . . 67
15.11 1D Consistency Checks . . . 67
15.12 A Simpler But Less Smooth 1D Alternative . . . 67
15.13 3D Case: Two Different Meanings Of "Constant Velocity" . . . 68
15.13.1 Constant Velocity Vector . . . 68
15.13.2 Constant Speed Along The Original 3D Path . . . 68
15.14 Decided 3D Interpretation For This Requirement . . . 68
15.15 3D Path-Preserving Strategy: Arc-Length Reparameterization . . . 69
15.16 Important Consequence In 3D . . . 70
15.16.1 Implementation Note: Where Numerical Methods Are Actually Needed . . . 70
15.17 3D Straight-Chord Alternative . . . 70
15.18 How The Start And End Positions May Be Specified In 3D . . . 70
15.18.1 Absolute Specification . . . 70
15.18.2 Percentage Specification Along The Path . . . 71
15.19 Transition Segments Around The Constant-Speed Part . . . 71
15.19.1 1D Transition Targets . . . 71
15.19.2 3D Transition Targets . . . 71
15.20 Baseline Algorithm For Requirement 12 . . . 72
15.21 Relation To The Data Model . . . 72
15.22 Worked Symbolic Considerations . . . 73
15.22.1 Example 1: 1D Exact Constant-Velocity Middle Segment . . . 73
15.22.2 Example 2: 3D Path-Preserving Constant-Speed Segment . . . 73
15.22.3 Example 3: 3D Straight-Chord Alternative . . . 73
15.23 Open Questions . . . 73
15.24 Related Documents . . . 74

16 Blending Segments **74**
16.1 Intro And Scope . . . 74
16.2 Symbols Used . . . 74
16.3 Why Blending Is Needed . . . 75
16.4 The Meaning Of "Without A Pass-Through Constraint" . . . 75
16.5 Local Blend Window . . . 75
16.6 1D Blending Problem . . . 76
16.6.1 Nominal Consecutive Segments . . . 76
16.7 Recovering The Corresponding Parameters . . . 76
16.8 Sampling The Boundary States For The Blend . . . 77
16.9 Building The Blend Segment In 1D . . . 77
16.10 Why Degree 9 Is A Natural Blend Reference . . . 77
16.11 Lower-Degree Analogues . . . 78
16.12 Choosing The Blend Duration . . . 78
16.13 Recommended 1D Baseline . . . 78
16.14 Absolute Vs Percentage Specification In 1D . . . 78
16.14.1 Absolute Specification . . . 78
16.14.2 Percentage Specification . . . 79
16.15 1D Blend Does Not Need To Pass Through The Nominal Join . . . 79
16.16 3D Blending Problem . . . 79
16.16.1 Geometric Meaning In 3D . . . 79
16.17 Recovering The Corresponding Parameters In 3D . . . 80
16.18 Sampling The Boundary States In 3D . . . 80
16.19 How The 3D Blend Is Actually Computed . . . 80
16.20 Why The 3D Blend Is Still Different From Purely Independent Axis Work . . . 81
16.21 Straight-Line Shortcut Vs Genuine 3D Blend . . . 81
16.22 Absolute Vs Percentage Specification In 3D . . . 81
16.22.1 Absolute Specification . . . 81

16.22.2 Percentage Specification . . . 81
16.23 Relation Between Blending And Constraints . . . 82
16.24 Baseline Blending Algorithm . . . 82
16.25 Relation To The Data Model . . . 82
16.26 Worked Symbolic Considerations . . . 83
16.26.1 Example 1: 1D Blend Around A Nominal Join . . . 83
16.26.2 Example 2: Degree-9 Blend . . . 83
16.26.3 Example 3: 3D Blend . . . 83
16.27 Open Questions . . . 83
16.28 Related Documents . . . 84

17 Trajectory Passing Through Constraint Points 84
17.1 Intro And Scope . . . 84
17.2 Symbols Used . . . 84
17.3 Why Requirement 13 Is Not Enough . . . 85
17.4 Exact Pass-Through In 1D . . . 85
17.4.1 Waypoint Sequence . . . 85
17.4.2 Why A Stop-And-Go Solution Is Undesirable . . . 86
17.4.3 Boundary Conditions At The Global Ends . . . 86
17.4.4 Interior Waypoint States . . . 86
17.5 Segment Representation In 1D . . . 87
17.5.1 One Segment Per Consecutive Pair . . . 87
17.5.2 Automatic Continuity Through Shared States . . . 87
17.6 Why The Full Problem Is Global . . . 87
17.6.1 Local Segment Solves Are Easy . . . 87
17.6.2 Determining The Interior States Is The Hard Part . . . 87
17.7 Three Families Of Global Strategy . . . 88
17.7.1 Prescribed Interior Derivatives . . . 88
17.7.2 Heuristic Interior Derivatives . . . 88
17.7.3 Optimization-Based Global Determination . . . 88
17.8 Recommended 1D Baseline For This Project . . . 88
17.9 Exact Pass-Through Versus Blending . . . 89
17.10 Segment Durations In 1D . . . 89
17.11 Lower-Degree Versions . . . 89
17.12 3D Exact Pass-Through . . . 89
17.12.1 Exact Waypoints In 3D . . . 89
17.12.2 Shared Waypoint States In 3D . . . 90
17.12.3 How The 3D Segments Are Computed . . . 90
17.12.4 Avoiding Zero Velocity In 3D . . . 90
17.13 Recommended 3D Baseline . . . 90
17.14 Why Requirement 14 Is More Constrained Than Requirement 13 . . . 91
17.15 Baseline Algorithm . . . 91
17.16 Relation To The Data Model . . . 91
17.17 Worked Symbolic Considerations . . . 92
17.17.1 Example 1: Three Waypoints In 1D . . . 92
17.17.2 Example 2: Difference From Ordinary Blending . . . 92
17.17.3 Example 3: 3D Exact Pass-Through . . . 93
17.18 Open Questions . . . 93
17.19 Related Documents . . . 93

1 Vision

The goal of this project is to enter in detail into the trajectory planning process, considering polynomial algorithms of different degree.

The degrees to be approached are, at the moment, three, five, seven and nine. Other degrees will be added over time if deemed useful during development (see `04-future-topics.md`).

The project will culminate in an **application** (web or desktop — see the open decision in `01-requirements.md`, item 16) that

analytically determines the trajectories discussed throughout the project. It is also intended as a personal learning experience in polynomial calculus and the techniques behind it. Everything produced should therefore be as educational as possible.

1.1 Outputs

Two outputs are expected:

1. **The application** — graphical output and data input, satisfying the technical specifications.
2. **A knowledge reference** — a readable document (or group of documents) / project log, capturing procedures, calculations, considerations, and theory as they are developed. Ideally this log records the steps taken and the theoretical/practical considerations behind each development decision.

1.2 Topic classification

Every requirement in `01-requirements.md` is tagged as one of:

- `[Learning]` — theoretical concepts and procedures aimed at deepening understanding of the topic.
- `[Technical]` — aspects directly involved in the implementation of the application.
- `[Learning & Technical]` — applies to both.

For every tagged item, a written report of the procedures (calculations, considerations) and algorithms used — and, when applicable, of the underlying theory — is expected.

1.3 General rules (apply throughout, unless a requirement says otherwise)

To avoid repeating the same clause on every requirement, the following rules are declared once here and apply globally:

- **Dimensionality**: every analysis, algorithm, or consideration is first developed in one-dimensional (1D) space. Once established, any differences arising when extending the same concept to three-dimensional (3D) space are noted explicitly. This progression (1D → note 3D differences) is the default for all requirements unless stated otherwise.
- **Numerical vs. symbolic**: unless numerical data is strictly necessary for the context, symbolic expressions are preferred over numerical examples.

2 Requirements

Tags: `[Learning]`, `[Technical]`, `[Learning & Technical]` — see `00-vision.md` for definitions. Where a requirement's dimensionality progression (1D → 3D) or symbolic-vs-numeric preference is not restated, the general rules in `00-vision.md` apply.

1. **`[Learning & Technical]`** Define the symbology to be adopted to clearly and correctly represent polynomial paths in one and three dimensions, including symbols for orienting a solid in three-dimensional space (the latter to be detailed in a later step — see `04-future-topics.md` → Orientation). Two-dimensional notation is intentionally omitted because it is considered a conceptual subset of the three-dimensional case. This is a starting point to be established before other points.
2. Regarding the polynomial of degree 9 (the first to be considered):
   - **`[Learning]`** Report the canonical form and the Horner form in the time domain; list pros and cons of the two forms.
   - **`[Learning]`** Consider the derivatives up to the sixth, starting from the position algorithm (velocity, acceleration, jerk, snap, crackle, pop); discuss the usefulness of using more or fewer derivatives for trajectory planning.
3. - **`[Technical]`** Report the mathematical procedure to calculate the coefficients of the polynomial, assuming all necessary boundary data has been specified.
   - **`[Learning]`** List the boundary conditions relevant to each polynomial degree considered in the project, distinguishing between necessary, possible, and redundant conditions for that degree.
4. **`[Learning]`** Report the analysis (pros and cons) of using an absolute period vs. a normalised period.
5. **`[Learning & Technical]`** Symbolic-vs-numeric preference — see general rule in `00-vision.md`.
6. **`[Learning & Technical]`** Dimensionality progression for items 2–5 — see general rule in `00-vision.md`.
7. **`[Technical]`** For a polynomial of degree 9, carry out the following analysis where relevant: procedure and algorithms to determine max/min position, max/min velocity, acceleration, jerk, and snap.

8. `[Technical]` Procedure and algorithms necessary to impose a velocity limit on a polynomial of degree nine.
9. `[Technical]` As item 8, but imposing a maximum acceleration.
10. `[Technical]` As item 8, but imposing a maximum jerk.
11. `[Learning & Technical]` Dimensionality progression for items 7–10 — see general rule in `00-vision.md`.
12. `[Learning & Technical]` If a constant-speed segment is required within a path originally defined as a degree-nine polynomial, describe the considerations and algorithms needed to satisfy this. The constant-speed segment is specified by its start and end positions within the original path, in absolute terms or percentages (arbitrary choice of convenience). Cover 1D first, then 3D (per general rule).
13. `[Technical]` Complex trajectories are normally composed of more than one segment (from two to several dozen). The connection between consecutive segments should be specified by: the starting point of the connection (in the segment preceding the common connection point) and the ending point of the connection (in the segment following it) — in absolute terms or percentages. Report the strategy and algorithms involved. Cover 1D first, then 3D (per general rule).
14. `[Learning & Technical]` The smooth-connection approach from item 13 works correctly except when the path must pass exactly through the endpoints of the specified segments, avoiding a "zero velocity" event. A smooth connection with precise transition points (the segment endpoints) is required instead. Assuming velocity, acceleration, jerk, and snap are zero at the start of the first segment and the end of the last segment, report the considerations and solution strategy for this case, in both 1D and 3D (per general rule).
15. `[Learning & Technical]` The output resulting from the above points should be summarized in a software application including a graphical representation of the calculated profiles, based on data acquired from an input interface (details to be provided during development). At the moment only the degree-nine polynomial has been considered, but the final version will extend the same considerations/analysis/calculations to degrees seven, five, and three (only the applicable ones per degree). The second output is the project log — a single or multiple readable document(s) reporting notes, considerations, strategies, and theoretical treatments involved in the application.
16. `[Technical]` — **DECISION: desktop application, C#, WPF.**
    - **Platform**: desktop application (not web).
    - **Language / toolchain**: C#, WPF for the UI, a charting library (e.g. LiveCharts2 / OxyPlot / ScottPlot) for graphical visualization of the computed profiles.
    - **Rationale**: no requirement in this document calls for multi-user access, remote sharing, or cross-device execution — the tool is a single-user local analysis application, so the web's main advantages don't apply here. A web stack would additionally require frontend/backend tooling unrelated to trajectory planning, conflicting with the constraint below. A desktop app keeps all calculation logic (coefficients, derivatives, constraints, blending) in one project and one language, which best matches the educational goal of understanding the theory rather than a client/server architecture.
    - Constraint (kept from the original brief): the purpose of the project is primarily to deepen knowledge of trajectory planning, not to learn new tools/languages unless strictly necessary for that goal.
    - Constraint (kept from the original brief): the project author must remain involved in and aware of the specific content of the project software — the educational aspect must not be sacrificed for development speed, even if this slows progress.
    - At this stage no hardware/actuator/kinematics application is considered — theory only (see `03-assumptions.md`), even though the natural extension is toward robotic manipulators.
    - **Revisit if**: a future requirement emerges for multi-user access, remote/cross-device use, or integration with an online service — none of these apply today.

3 Data Model

This chapter summarizes the data entities involved in the project. Orientation is **not** considered yet in these models (see `04-future-topics.md` → Orientation); it will be added later.

This is a reference example for the data potentially involved in the project — it will be revised as the project progresses, as needed.

3.1 Points

```
Point1D {
```

```
  x;   // mono-dimensional position value of a single point
}

Point3D {
  x;   // three-dimensional x position value of a single point
  y;   // three-dimensional y position value of a single point
  z;   // three-dimensional z position value of a single point
}
```

3.2 Segments and Trajectories

A segment of a complex trajectory is itself a trajectory:

```
Segment1D {
  startPoint;          // Point1D
  endPoint;            // Point1D
  polynomialDegree;    // chosen among 3, 5, 7, 9 (and eventually other values)
  constraints[];        // settings of constraints applied to the segment
}

Segment3D {
  startPoint;          // Point3D
  endPoint;            // Point3D
  polynomialDegree;    // chosen among 3, 5, 7, 9 (and eventually other values)
  constraints[];        // settings of constraints applied to the segment
}

Trajectory1D {
  Segment1D[nn];              // array of Segment1D elements
  segmentEndpointRespect;     // yes/no - if yes, segment blending settings become irrelevant
}

Trajectory3D {
  Segment3D[nn];              // array of Segment3D elements
  segmentEndpointRespect;     // yes/no - if yes, segment blending settings become irrelevant
}
```

3.3 Polynomial and boundary conditions

```
Polynomial {
  degree;              // chosen among three, five, seven, nine, etc.
  coefficients;
  boundaryConditions;
}

BoundaryConditions {
  necessaryConditions[];
  possibleConditions[];
  redundantConditions[];
}
```

`BoundaryConditions` is crucial to the calculation of polynomial profiles of any degree. Its content depends on the polynomial degree being treated: each degree has a set of necessary conditions to determine the polynomial, a set of possible conditions that may be useful in broader formulations, and a set of redundant conditions that exceed what is needed for that degree.

3.4 Constraints and Blend

```
Constraints {
  constraintType;              // time, velocity, acceleration, jerk, snap, etc. (if applicable)
  constraintValue;              // value referred to the selected constraintType
```

```
  constantVelocitySelection;     // yes/no - activates the constant-velocity part of the segment
  constantVelocityValue;         // applied if constantVelocitySelection = yes
  constantVelocityInitialPos1D;  // Point1D - start of the constant-velocity part of the segment in 1D
  constantVelocityEndPos1D;      // Point1D - end of the constant-velocity part of the segment in 1D
  constantVelocityInitialPos;    // Point3D - start of the constant-velocity part of the segment
  constantVelocityEndPos;        // Point3D - end of the constant-velocity part of the segment
}

Blend {
  nextSegmentBlendingPos1D;      // Point1D - where blending with the next segment starts in 1D
  previousSegmentBlendingPos1D;  // Point1D - where blending with the previous segment ends in 1D
  nextSegmentBlendingPos;        // Point3D - where blending with the next segment starts
  previousSegmentBlendingPos;    // Point3D - where blending with the previous segment ends
}
```

> **Design note (confirmed distinction)**: `Constraints` and `Blend` are deliberately distinct concepts, kept separate: - **`Constraints`** describes kinematic limits/settings applied *within* a single segment (velocity/acceleration/jerk/snap limits, and the optional constant-velocity sub-part of the segment, via the dedicated 1D or 3D positions). - **`Blend`** describes the geometric continuity *between* adjacent segments — the raccordo (smoothing) at the junction points, via the dedicated 1D or 3D blending positions.
>
> Relevant to requirements 12–14 in `01-requirements.md`.

3.5 State (conceptual view)

The `State` element is an elementary entity of the trajectory concept: it specifies an endpoint (initial or final, relative to a single segment) and its relevant details.

```
State1D {
  position; velocity; acceleration; jerk; snap;
}
```

The same concept applies to `State3D`. `State` may be more relevant than `Point` when approaching the framework's motion-planning task.

3.6 How the entities relate

- **`Constraints`** and **`Blend`** are project-specific requirements — their settings/values are internal parts of the whole project (see open note above).
- **`Trajectory`** is the top-level container that encompasses the entire data structure (all elements above, in terms of selections and values) leading to the determination of the overall movement profile — the purpose of the project.

4 Assumptions

1. Only theoretical trajectory generation.
2. No actuator dynamics.
3. No kinematic constraints.
4. Single-axis analysis before multi-axis analysis.
5. Symbolic treatment preferred over numerical examples.

5 Future Topics

Not in the current scope, but tracked for later consideration.

5.1 Orientation

- Euler angles.
- Quaternions.
- Rotation matrices.

5.2 Time scaling

- Trajectory stretching.
- Trajectory compression.

5.3 Multi-axis synchronization

- Synchronous end-time.
- Master axis.
- Slave axes.

5.4 Path vs Trajectory

- Path = geometric definition.
- Trajectory = path + time law.

5.5 Numerical Stability

- Conditioning.
- Stability.
- Floating point errors.

5.6 Implementation notes carried over from the original brief

- Polynomial degrees other than three, five, seven and nine may be added if deemed useful during project development.
- Application type decided: desktop application in C# with WPF (see `01-requirements.md`, item 16).
- Readable documentation for learning is a hard requirement (the author's own stated need), not optional polish.
- Open question: derivatives usefulness — up to which degree is it actually useful to compute/use them?
- Orientation of a solid in three-dimensional space (see Orientation above).
- Technical tools for app development decided: C# for the calculation/application code, WPF for the UI, and a charting library chosen during implementation — see `01-requirements.md`, item 16.

6 Success Criteria

A project is considered successful if:

- The mathematical theory is completely documented.
- Every algorithm is explained.
- Every software decision is justified.
- The generated application can calculate polynomial trajectories.
- The generated application can visualize the results graphically.
- A complete educational project log is produced.
- The author can understand the theory and implementation process.

Suggestion (not part of the original brief): the criteria above are qualitative. Now that the technical stack (item 16) is decided (C#/WPF), a short "Measurable criteria" section could be added here — for example, "the app computes and plots a degree-9 profile from 10 boundary conditions in under 1s".

7 Theory Library Roadmap

This is the working checklist for the "personal library" of theory/technical documents to be produced together with Claude, one requirement-cluster at a time. Each entry maps to the requirement(s) it covers in `01-requirements.md`, so coverage can be checked off as documents are produced.

Suggested per-document template (for consistency across the library): short intro / scope → symbols used (referencing `SymbolsAndNomenclature`) → theory / derivation → algorithm or procedure → worked considerations (symbolic, per `03-assumptions.md`) → open questions / links to related documents.

7.1 Format convention

- **Source format**: every document in this library is written in **Markdown** (`.md`), consistent with the specification files (`00`–`07`). `08-ui-first-prototype.md` was added later, once the theory library reached M2, as the concrete specification for the first GUI milestone — see `07-implementation-plan.md`.
- **Mathematical notation**: formulas and equations are written as LaTeX syntax within the Markdown: inline math uses single delimiters like `$x(t)$`, while display equations on their own line use double delimiters like `$$x(t) = \sum_{i=0}^{9} c_i t^i$$`. Math delimiters are never wrapped in backticks. This keeps documents renderable in-editor (MathJax/KaTeX) and diffable in version control. Polynomial coefficients are denoted c_i, not a_i, to avoid colliding with acceleration ($a(t)$, a_0, a_f) — see `SymbolsAndNomenclature.md`.
- **Reading / archival output**: when a readable/printable version of the library is needed, it is generated from the Markdown sources via `pandoc` (LaTeX engine) into PDF — the Markdown+LaTeX-inline files remain the single source of truth; PDF is a generated artifact, never edited directly.
- **Not used as a working/source format**: LaTeX (`.tex`) as primary source, Word (`.docx`), ODT, plain `.txt`. See discussion in chat for the rationale.

7.2 Pre-publication verification checklist

Before considering any document in this library "finished", verify it with the following steps:

1. Compile it: `pandoc <file>.md -o <file>.pdf --pdf-engine=xelatex -V mainfont="<a system font>" -V geometry:margin=2.5cm`
2. **Check inline math specifically** (formulas embedded in running text, not standalone display equations on their own line) — this is where syntax errors are most likely to hide, since display-equation blocks tend to be visibly broken (and so get caught immediately) while inline math can silently render as literal text instead of a formula.
3. Confirm backticks are only used around genuine code/filename references, never around `$...$` math delimiters — inline math must be single `$...$`, with no surrounding backticks (backtick-wrapped `$$...$$` renders as literal text instead of a formula).
4. If the PDF build fails with a missing LaTeX package (e.g. `lmodern.sty` not found), install the missing package via the local TeX distribution (MiKTeX installs it automatically on Windows; `tlmgr install <package>` on TeX Live) — or work around it by forcing a system font with `--pdf-engine=xelatex -V mainfont="<font name>"`.
5. **In tables, never write a multi-symbol list as a single inline math block** (e.g. `$x_0, x_f, v_0, v_f$`) — LaTeX cannot line-wrap inside one math box, so it silently overflows into the next column instead of wrapping. Write each symbol as its own inline math, comma-separated in plain text (e.g. `$x_0$, $x_f$, $v_0$, $v_f$`), so the cell can wrap normally.
6. **Respect heading hierarchy — never place two consecutive headings at the same level when the second is logically a subsection of the first** (e.g. a `##` "1D Blending Problem" section immediately followed by a `##` "Nominal Consecutive Segments" that is actually part of it). The second heading should be one level deeper (`###`).

#	Document	Covers (requirements)	Status
1	`SymbolsAndNomenclature`	Req. 1 — symbology for 1D/3D polynomial paths and solid orientation (Euler angles, quaternions, rotation matrices — notation reserved now, full treatment later per `04-future-topics.md`)	Draft v1
2	`NinthDegreePolynomialTheory`	Req. 2 — canonical & Horner form, derivatives up to the 6th (velocity → pop), pros/cons discussion	Draft v1
3	`BoundaryConditions`	Req. 3 (learning part) — full list of boundary conditions needed to determine the polynomial	Draft v1

#	Document	Covers (requirements)	Status
4	`PolynomialCoefficientDetermination`	Req. 3 (technical part) + Req. 4 — coefficient-calculation procedure; absolute vs. normalised period analysis	Draft v1
5	`TrajectoryConstraints`	Req. 7–10 — max/min position/velocity/acceleration/jerk/snap analysis; imposing velocity/acceleration/jerk limits	Draft v1
6	`ConstantVelocitySegment`	**Req. 12** — constant-speed segment within a degree-9 path (start/end position, absolute or percentage)	Draft v1
7	`BlendingSegments`	Req. 13 — connecting consecutive segments without a pass-through constraint at the join	Draft v1
8	`TrajectoryPassingThroughConstraintPoints`	Req. 14 — smooth connection with precise transition points (zero velocity/acceleration/jerk/snap at path start/end)	Draft v1

Note on item 6: this document was not in the original list of documents discussed — it covers requirement 12 (constant-velocity segment), which is distinct both from `TrajectoryConstraints` (general kinematic limits) and from `BlendingSegments`/`TrajectoryPassingThroughConstraintPoints` (continuity *between* segments, per the `Constraints` vs. `Blend` distinction in `02-data-model.md`). It is placed here because it logically follows the general constraints treatment and precedes the inter-segment blending topics.

7.3 Out of scope for this roadmap (tracked elsewhere)

- Requirement 15 (the application itself, graphical interface, extension to degrees 3/5/7) is not a theory document — it is the implementation phase in `src/`. This has since been formalized in `07-implementation-plan.md` (consolidated implementation decisions) and `08-ui-first-prototype.md` (the M2 UI specification), and `src/` now implements the degree-9 core engine (M1), degrees 3/5/7 (M3), constraint verification and duration search (M4), the constant-velocity segment (M5, 1D only), blending (M6, 1D only), a provisional pass-through solver (M7 — see `08-ui-first-prototype.md`'s note that it is not yet the exact minimum-snap formulation), and a partial 3D extension (M8 — component-wise wrapping only; 3D constant velocity and blending are not yet implemented).
- Requirements 6 and 11 (1D → 3D progression) are not separate documents — per the general rule in `00-vision.md`, each document above covers 1D first and notes 3D differences inline.
- Future Topics (orientation in full, time scaling, multi-axis sync, numerical stability) remain in `04-future-topics.md` until promoted to their own roadmap entries.

8 Implementation Plan

8.1 Intro And Scope

This document is the bridge between the theory library (`SymbolsAndNomenclature` through `TrajectoryPassingThroughConstraintP` tracked in `06-theory-library-roadmap.md`) and the software implementation phase in `src/` (Requirement 15).

Its purpose is to consolidate the implementation-oriented Open Questions scattered across the eight theory documents — many of which turned out to be the same underlying decision restated in different words — into a single set of resolved (or deliberately deferred) decisions, before any C#/WPF code is written. This mirrors, for the theory→code transition, the same discipline already applied when the theory library itself was planned in `06-theory-library-roadmap.md`.

This document does not re-derive any theory. It only decides how the already-validated theory gets translated into software structure.

8.2 Consolidated Decisions

Each decision below traces back to one or more Open Questions in the theory library, listed for reference.

8.2.1 Decision 1 — Segment/Blend/Transition Duration Strategy

Open Questions consolidated from: `TrajectoryConstraints` (Q2), `BlendingSegments` (Q1), `TrajectoryPassingThroughConstraintP` (Q2).

There is no single duration strategy for the whole project — the right default differs by context:

- **General kinematic limits** (`TrajectoryConstraints`): already decided in that document — **recompute-and-verify**. Choose an initial duration, verify the active constraint, increase duration and recompute if violated, repeat.
- **Constant-velocity segment** (`ConstantVelocitySegment`): not a duration-strategy case at all — T_{cv} is fully determined by the distance/speed formula already established there. No decision needed.
- **Blending** (`BlendingSegments`): **default = inherited from the removed original portions** of the neighboring segments. Simplest to implement first, geometrically intuitive.
- **Exact pass-through** (`TrajectoryPassingThroughConstraintPoints`): **default = derived from waypoint spacing** (segment duration proportional to the distance between consecutive waypoints).

Architecture note: implement duration selection behind a single small abstraction (e.g. an `IDurationStrategy` per context, or a shared enum `{ UserSpecified, AutoDerived }`) reused across all four contexts, rather than four separate ad-hoc mechanisms. All defaults above remain replaceable later by a constrained search (duration as an optimization variable) without changing calling code.

8.2.2 Decision 2 — Boundary Condition / Waypoint State Representation

Open Questions consolidated from: `BoundaryConditions`, `PolynomialCoefficientDetermination` (Q3), `TrajectoryPassingThrough` (Q3).

A single, degree-agnostic representation is adopted instead of per-degree typed structures:

```
enum DerivativeOrder { Position, Velocity, Acceleration, Jerk, Snap }

record BoundaryCondition {
    DerivativeOrder Order;
    double Value;              // Point3D / Vector3D in 3D, per Decision-by-context
}
```

`necessaryConditions[]`, `possibleConditions[]`, and `redundantConditions[]` (per `02-data-model.md`) are plain lists of `BoundaryCondition`. Which derivative orders are required for a given degree (per `BoundaryConditions.md`'s classification table) is implemented as a separate validation lookup, not as separate classes per degree — one representation for degrees 3, 5, 7, and 9 alike.

The same `BoundaryCondition` record is reused for shared interior waypoint states in `TrajectoryPassingThroughConstraintPoints` — a waypoint state is simply a small collection of `BoundaryCondition` values at one point.

This is a deliberately minimal starting representation — see *Open Points Carried Forward* below for its planned refinement trigger.

8.2.3 Decision 3 — Absolute Vs Percentage Specification

Open Questions consolidated from: `ConstantVelocitySegment`, `BlendingSegments` (Q2), `BoundaryConditions` (UI open question).

DECISION: positions and reference points (blend points, constant-velocity sub-segment endpoints, etc.) are specified in **absolute** terms relative to the segment endpoints. Percentage-based specification is not implemented in this baseline.

Accepted trade-off: if a segment's original endpoints are later moved, absolute reference points do not automatically follow — they would need to be re-entered. This is accepted given the project's scope (segments are not expected to be moved frequently once set up).

8.2.4 Decision 4 — Numerical Strategy For Non-Closed-Form Cases

Open Questions consolidated from: `TrajectoryConstraints` (Q1), `ConstantVelocitySegment`'s implementation note.

DECISION: implement the strategy fully — both branches, not a single simplified path:

- **Closed-form branch**: used whenever the theory guarantees it (single-axis constant-velocity segments, all Horner/canonical polynomial evaluation, degree ≤ 4 stationarity equations where applicable).
- **Numerical branch**: used only where the theory shows no closed form exists (root extraction for stationarity polynomials of degree ≥ 5 in `TrajectoryConstraints`; arc-length integration and inversion for genuinely curved multi-axis constant-velocity segments in `ConstantVelocitySegment`).

Library choice: **Math.NET Numerics** (mature, free, well-documented .NET library) for root-finding (e.g. Brent's method) and numerical integration, instead of a hand-written solver. This keeps the project's educational focus on trajectory planning rather than on numerical-methods implementation details, consistent with the constraint in `01-requirements.md` item 16 against learning new tools/techniques not strictly necessary to the project's actual goal.

Implementation should branch on the single-axis/straight-subpath detection established in `ConstantVelocitySegment` to avoid paying the numerical cost when the closed form applies.

8.2.5 Decision 5 — Smoothness Criterion For Exact Pass-Through

Open Question consolidated from: `TrajectoryPassingThroughConstraintPoints` (Q1).

DECISION: **minimum-snap** is adopted as the sole smoothness criterion for determining shared interior waypoint derivatives, consistent with the special structural role of snap already established in `NinthDegreePolynomialTheory`.

Architecture note: implement the criterion behind a small interface (e.g. `ISmoothnessCriterion`) rather than a hard-coded cost function, so an alternative (e.g. minimum-jerk, computationally cheaper but less smooth) could be substituted later without restructuring the surrounding waypoint-solving code.

8.2.6 Decision 6 — Polynomial Evaluation Form

Open Question consolidated from: `NinthDegreePolynomialTheory` (Q1).

DECISION: **Horner form** is used for all repeated numerical evaluation (sampling a segment at many time instants for plotting). Canonical form remains the reference form for symbolic derivation and documentation, per the distinction already established in `NinthDegreePolynomialTheory`.

8.3 Deferred Decisions

These remain open on purpose — deciding them now, without the concrete context they depend on, would risk a premature and possibly wrong choice.

8.3.1 Deferred — UI Exposure Choices

Open Questions consolidated from: `NinthDegreePolynomialTheory` (Q3), `BoundaryConditions` (UI open question), `BlendingSegments` (Q3).

Deferred to the UI design phase itself:

- which derivative levels are shown by default in the charts, and which are optional;
- whether 3D boundary conditions are entered axis-by-axis or in vector form;
- whether Requirement 13 and Requirement 14 are exposed as two separate modes or as one continuity tool with a pass-through toggle.

8.3.2 Deferred — Charting Library

Carried over from `01-requirements.md` item 16, which explicitly left this choice for implementation time.

Candidates identified at planning time were: LiveCharts2, OxyPlot, ScottPlot.

Status update after M2 implementation: this decision has now been resolved by `08-ui-first-prototype.md` and the implemented WPF prototype. The chosen charting library is **ScottPlot**.

8.4 Open Points Carried Forward

- **Decision 2's representation is a deliberate starting point, not a final design.** It should be revisited once real GUI and data-persistence requirements exist — the trigger for revisiting is the start of UI design work, not a fixed milestone number.
- **Decision 1's defaults are replaceable** by a constrained-search strategy (duration as an optimization variable under active constraints) if the fixed defaults prove insufficient once constraint verification (`TrajectoryConstraints`) is exercised against real blend/pass-through segments.

8.5 Project Structure

Two projects, to keep calculation logic testable independently of the UI, consistent with the educational goal of understanding the theory rather than the framework:

```
src/
  TrajectoryPlanner.Core/         # pure calculation library, no UI
    Symbols/                      # DerivativeOrder, shared constants
    Polynomials/                  # coefficients, canonical + Horner
    BoundaryConditions/           # BoundaryCondition record
    Constraints/                  # extrema, limits, duration search
    ConstantVelocity/             # closed-form + numerical branches
    Blending/                     # blend segment construction
    PassThrough/                  # global minimum-snap solve
  TrajectoryPlanner.App/          # WPF project, references Core
  TrajectoryPlanner.Core.Tests/   # unit tests against Core only
```

This structure lets each theory document map to one folder/namespace in `Core`, keeping the traceability the project has maintained throughout the theory library.

8.6 Suggested Milestone Sequence

Following `03-assumptions.md` item 4 (single-axis analysis before multi-axis analysis), 1D is implemented and validated end-to-end before the 3D extension is added as a wrapping layer that reuses the same component-wise logic three times (per the baseline already established in `TrajectoryConstraints`).

1. **M1 — Core degree-9 engine (1D)** (`PolynomialCoefficientDetermination`, `NinthDegreePolynomialTheory`): `BoundaryCondition` model, coefficient determination, canonical and Horner evaluation, derivatives up to the 6th.
2. **M2 — Minimal WPF shell**: boundary-condition input, single-segment plot. This is where the charting library decision is made.
3. **M3 — Lower-degree support** (`BoundaryConditions`): extend M1 to degrees 3, 5, 7 per the per-degree table.
4. **M4 — Constraints** (`TrajectoryConstraints`): extrema analysis and limit verification, recompute-and-verify duration search.
5. **M5 — Constant-velocity segment** (`ConstantVelocitySegment`): closed-form branch first, then the numerical branch.
6. **M6 — Blending** (`BlendingSegments`).
7. **M7 — Exact pass-through** (`TrajectoryPassingThroughConstraintPoints`): global minimum-snap waypoint solve.
8. **M8 — 3D extension**: component-wise wrapping of M1–M7 into `Segment3D`/`Trajectory3D`.

8.7 Relation To The Data Model

`02-data-model.md` remains the conceptual reference; this document adds the concrete representation decided in *Decision 2* for `BoundaryConditions`' array contents, and confirms `Constraints/Blend` map directly to the `Constraints/` and `Blending/` folders above. No further data-model revision is required to start M1.

8.8 Appendix — Stable Facts Emerged During Initial Implementation

The following points were decided implicitly during the first implementation pass in `src/` and are now recorded here so they do not remain only in chat history or in code:

8.8.1 A1 — Decision 2 Was Refined Into Typed 1D/3D Boundary Conditions

Decision 2 originally proposed one minimal, degree-agnostic `BoundaryCondition` sketch as a starting point.

The implemented baseline refines that sketch into two typed records:

- `BoundaryCondition1D`
- `BoundaryCondition3D`

Both carry:

- `DerivativeOrder`
- `BoundaryEndpoint` (`Start` / `End`)

and then:

- a scalar `double` value in `1D`;
- a `Point3D` value in `3D`.

This is a post-plan implementation refinement, not a theory change. Its purpose is to keep dimensional typing explicit while also making endpoint identity part of the condition itself rather than leaving it implicit.

8.8.2 A2 — The Duration Abstraction Was Concretized

Decision 1 proposed a small shared abstraction for duration handling. The initial implementation has concretized that idea as:

```
DurationMode { UserSpecified, AutoDerivedProportional, AutoDerivedFromConstraints }
```

with strategy implementations behind `IDurationStrategy1D`.

This makes one earlier chat-level idea explicit: the project does not use a single monolithic duration rule, but a small strategy family that already separates:

- directly entered durations;
- automatically distributed durations;
- constraint-aware duration search.

8.8.3 A3 — Trajectory Construction Uses Builder-Style Services

The initial implementation has adopted builder-style orchestration services as the main application-facing assembly layer:

- `TrajectoryBuilder1D`
- `TrajectoryBuilder3D`

These builders centralize the composition steps that otherwise would be scattered across the UI layer:

- add single segments;
- create rest-to-rest segments;
- build pass-through trajectories from waypoint arrays;
- insert blends between adjacent segments.

This is now a stable architectural fact of the implementation baseline.

8.8.4 A4 — The Test Stack Is xUnit And Remains Core-Only

The testing project `TrajectoryPlanner.Core.Tests` has been implemented with **xUnit**.

This records two practical implementation choices that were previously only implicit in `src/`:

- **test framework**: xUnit;
- **test scope**: tests target `TrajectoryPlanner.Core` only, not the WPF layer.

This remains fully consistent with the original separation principle of keeping mathematical logic testable independently from the UI.

8.8.5 A5 — Milestone M2 Evolved Into A Chart-First Prototype Layout

The original milestone sketch described a minimal shell with boundary-condition input and a single-segment plot.

The implemented and now stabilized M2 baseline is more specific:

- the application uses a **chart-first layout**, with inputs and reports in a left control rail;

- the visualization area is the dominant region of the window;
- the plotted quantities are displayed as **stacked plots with independent Y scaling** rather than one compressed shared-axis plot;
- the window uses the project icon `IconsAndGraphics/calculus.png`;
- segment management includes both **add** and **remove selected segment**, while preserving at least one segment at all times.

These are implementation-level refinements of M2, not changes to the underlying theory scope.

8.9 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `NinthDegreePolynomialTheory.md`
- `BoundaryConditions.md`
- `PolynomialCoefficientDetermination.md`
- `TrajectoryConstraints.md`
- `ConstantVelocitySegment.md`
- `BlendingSegments.md`
- `TrajectoryPassingThroughConstraintPoints.md`
- `02-data-model.md`
- `01-requirements.md` (item 16)

9 UI First Prototype

9.1 Intro And Scope

This document specifies the minimal graphical interface needed to exercise the `TrajectoryPlanner.Core` engine end to end, concretizing Milestone M2 of `07-implementation-plan.md`.

This is a **validation testbed**, not the final application UI. Its only purpose is to let the project author input segment data, apply constraints/constant-velocity/blend settings, and see numeric and graphical results — so the boundaries stay deliberately tight, to avoid features creeping in beyond what is needed to validate the Core.

Scope for this prototype: 1D only, consistent with `03-assumptions.md` item 4 (single-axis analysis before multi-axis analysis). 3D UI is explicitly out of scope here (see *Resolution Of Open Points* below).

Requirement coverage: this prototype covers Requirement 13 (`BlendingSegments`) only. Requirement 14 (`TrajectoryPassingThrou` exact pass-through) is deliberately excluded — see *Resolution Of Open Points* for the rationale.

9.2 Operational Flow

1. Create a segment (segment count starts at one, increments via a button once the current segment's data is complete, and the currently selected segment tab can be removed through a dedicated button).
2. For the active segment, choose the polynomial degree and enter the boundary conditions required for that degree.
3. Optionally select one or more active constraints and enter their values.
4. Optionally enable a constant-velocity stretch within the segment.
5. If at least two segments are defined, optionally set blend values connecting to the previous/next segment.
6. Compute.
7. Visualize: numeric report and chart.

9.3 Input Requirements

A single tab-based panel per segment, with a segment-number indicator, a button to add a new segment, and a button to remove the currently selected segment. The remove button is disabled when only one segment exists, so the prototype always keeps at least one editable segment. After a removal, segment tabs are renumbered sequentially. Each segment tab contains:

- **Degree**: dropdown, values 3 / 5 / 7 / 9.
- **Boundary conditions**: the necessary conditions for the selected degree (per `BoundaryConditions.md`'s classification table), each with its value field. The set of fields shown updates when the degree changes.

- **Constraints**: the full constraint list is always shown; each entry is individually selectable (checkbox) with its own value field, and **multiple constraints may be active at once** on the same segment, per the multi-constraint baseline already established in `TrajectoryConstraints.md`:
  - period / duration;
  - maximum velocity;
  - maximum acceleration (shown only if the selected degree supports it);
  - maximum jerk (shown only if the selected degree supports it);
  - maximum snap (shown only if the selected degree supports it).
- **Constant-velocity stretch**: a toggle, and when enabled, the start and end of the stretch, expressed relative to the whole segment (the interval between the segment's start and end position from its boundary conditions) — absolute values, per Decision 3 in `07-implementation-plan.md`.
- **Constant speed value**: when the constant-velocity stretch is enabled, the user also enters the speed value for that stretch explicitly.
- **Blend values**: shown only when at least two segments are defined — initial blend value (toward the previous segment) and final blend value (toward the next segment), relative to the whole segment.

9.4 Output Requirements

Two distinct outputs, produced after Compute:

- **Numeric report**: a table containing the polynomial coefficients and the maximum/minimum values of position, velocity, acceleration, jerk, and snap — limited to the quantities actually computable for the chosen degree.
- **Chart area**: a large visualization area occupying the dominant portion of the application window.
- **Per-quantity plots**: instead of compressing all curves into a single shared Y-axis plot, each computable quantity (position, velocity, acceleration, jerk, snap) is shown in its own vertically stacked plot with an independent Y scale, so each plot fully represents its own minimum and maximum values.
- **Cursor readout**: a shared time cursor across the stacked plots, with a numeric table beside the plots showing the value of each visible quantity at the cursor's current position.

9.5 Explicitly Out Of Scope For Now

- Project save/load.
- Refined chart styling.
- Advanced analysis tools (e.g. differential cursors comparing two points) — only the single value-readout cursor described above is included.
- 3D input and visualization.
- Exact pass-through (Requirement 14) — see rationale below.

9.6 Resolution Of Open Points From 07-implementation-plan.md

- **Default derivatives shown in charts**: **RESOLVED** — all derivatives applicable to the chosen degree are shown. The implemented prototype presents them as stacked plots with shared time inspection rather than compressing them into one shared Y-axis plot (see *Output Requirements*).
- **3D boundary condition input (axis-by-axis vs. vector)**: **remains open**, deferred to when 3D UI work begins (Milestone M8) — out of scope for this 1D-only prototype.
- **Unified vs. separate tool for Blending (Req. 13) and Exact Pass-Through (Req. 14)**: **remains open**, and for a reason beyond UI design: the current pass-through solver (`PassThroughSolver1D.cs`) is still a provisional finite-difference-style heuristic, not the exact global minimum-snap formulation promised by `TrajectoryPassingThroughConstraintPoints.md`. Designing UI for that requirement now would risk having to redesign it once the solver reaches its final form. This prototype therefore covers Blending only; Exact Pass-Through is added to the UI once the underlying solver is completed.

9.7 Charting Library

DECISION: **ScottPlot**.

ScottPlot was confirmed and adopted for the prototype because:

- it supports interactive time inspection cleanly;
- it is lightweight and free;

- it allows a large stacked-plot layout without additional framework complexity.

This decision is no longer provisional.

9.8 Implemented Clarifications

The following UI details were decided during the prototype implementation and are now fixed in this document:

- **Layout priority**: the chart area is the primary workspace. Inputs and reports are placed in a narrower left-side control/report column so visualization keeps most of the window width and height.
- **Optional controls**: constant-velocity and blend inputs are grouped as optional shape controls and may be collapsed when not needed, reducing wasted space.
- **Segment management**: the prototype exposes both add and remove actions at window level; removal always applies to the currently selected tab and is disabled when only one segment remains.
- **Window icon**: the application window uses `IconsAndGraphics/calculus.png`.

9.9 Relation To 07-implementation-plan.md

This document is the concrete specification for Milestone M2. It resolves one of the two items under *Deferred — UI Exposure Choices* (default derivatives shown) and narrows the other two to an explicit scope boundary (3D deferred to M8; Exact Pass-Through deferred to when its solver is complete) rather than leaving them open-ended. It also records that the *Deferred — Charting Library* decision has now been finalized as ScottPlot.

9.10 Related Documents

- `07-implementation-plan.md`
- `BoundaryConditions.md`
- `TrajectoryConstraints.md`
- `ConstantVelocitySegment.md`
- `BlendingSegments.md`
- `TrajectoryPassingThroughConstraintPoints.md`
- `06-theory-library-roadmap.md`

10 Symbols And Nomenclature

10.1 Intro And Scope

This document defines the notation used throughout the theory library and, later, in the software implementation notes. Its purpose is to remove ambiguity before entering derivations, algorithms, and design decisions.

The project works with polynomial trajectories in 1D first and then extends the same concepts to 3D when needed. The 2D case is intentionally omitted because it is considered a direct conceptual subset of the 3D case.

Orientation is included here only as reserved notation. Full theoretical treatment of orientation is explicitly postponed to a later phase.

10.2 Guiding Conventions

- Time-dependent quantities are written as functions of time, for example $x(t)$.
- Scalar quantities are written in plain italic notation, for example $x(t)$, $v(t)$, T.
- Three-dimensional vectors are written in bold, for example $\mathbf{p}(t)$.
- Initial and final values use subscripts 0 and f, for example x_0, x_f, $\mathbf{p}_0$, $\mathbf{p}_f$.
- Segment indices use subscripts k, for example $x_k(t)$.
- Normalized time is written as τ, while absolute time is written as t.

10.3 Time Variables

The following symbols are used for time:

- t: absolute time variable.
- t_0: initial time.

- t_f: final time.
- $T = t_f - t_0$: motion duration.
- τ: normalized time variable, usually defined as

$$\tau = \frac{t - t_0}{T}, \qquad \tau \in [0, 1].$$

This distinction is useful because some derivations are clearer in absolute time, while others are more compact in normalized time.

10.4 Position And Trajectory In 1D

In one dimension, the trajectory is represented by a scalar position law:

$$x(t)$$

For a polynomial of degree n, the canonical form is:

$$x(t) = \sum_{i=0}^{n} c_i t^i$$

where:

- c_i are the polynomial coefficients;
- n is the polynomial degree.

Coefficient notation vs. acceleration notation: the polynomial coefficients are denoted c_i rather than a_i. This is a deliberate departure from the more common textbook choice, made specifically for this project: `Boundary Conditions In 1D` below reserves a_0, a_f for the initial and final *acceleration*, and `NinthDegreePolynomialTheory` writes the acceleration profile itself as $a(t)$. Using a_i for the coefficients as well would make expressions such as $a(t) = 2a_2 + 6a_3 t + \ldots$ ambiguous at a glance — the same letter would denote both the function and its own coefficients in the same line. Normalized-time coefficients keep their existing symbol α_i (already distinct, see `PolynomialCoefficientDetermination`). This convention applies to every document in the library from this point on.

When the project specifically refers to the ninth-degree polynomial, the notation becomes:

$$x(t) = c_0 + c_1 t + c_2 t^2 + \cdots + c_9 t^9$$

The corresponding Horner form is written as:

$$x(t) = (((((((((c_9 t + c_8) t + c_7) t + c_6) t + c_5) t + c_4) t + c_3) t + c_2) t + c_1) t + c_0)$$

This document only fixes the notation. The analytical comparison between canonical and Horner form belongs to `NinthDegreePolynomialTheory`.

10.5 Derivatives In 1D

The project adopts the following derivative chain:

- position: $x(t)$
- velocity: $\dot{x}(t)$
- acceleration: $\ddot{x}(t)$
- jerk: $x^{(3)}(t)$
- snap: $x^{(4)}(t)$
- crackle: $x^{(5)}(t)$
- pop: $x^{(6)}(t)$

For quick reading, the shorthand notation below is also accepted when useful:

$$v(t) = \dot{x}(t), \qquad a(t) = \ddot{x}(t), \qquad j(t) = x^{(3)}(t), \qquad s(t) = x^{(4)}(t)$$

where:

- $v(t)$ denotes velocity;
- $a(t)$ denotes acceleration;
- $j(t)$ denotes jerk;
- $s(t)$ denotes snap.

For higher derivatives beyond snap, the explicit derivative notation $x^{(5)}(t)$ and $x^{(6)}(t)$ is preferred because it is less ambiguous than introducing too many extra letters.

10.6 Boundary Conditions In 1D

A boundary condition is a value imposed at the start or end of a segment. The standard notation is:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f,\ s_0,\ s_f$$

These symbols mean:

- x_0, x_f: initial and final position;
- v_0, v_f: initial and final velocity;
- a_0, a_f: initial and final acceleration;
- j_0, j_f: initial and final jerk;
- s_0, s_f: initial and final snap.

Not every polynomial degree requires the same set of boundary conditions. In later documents, boundary conditions will always be classified as:

- necessary;
- possible;
- redundant.

This classification depends on the polynomial degree under study.

10.7 State Notation

When it is useful to describe the motion condition at a single endpoint, the project uses the concept of state.

In 1D, a state can be written as:

$$\mathcal{S}_{1D} = (x, v, a, j, s)$$

or, if endpoint notation is needed:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = \left(x_f, v_f, a_f, j_f, s_f\right)$$

The symbol $\mathcal{S}$ (calligraphic) is used here as a compact state container. It is deliberately distinct, both visually and semantically, from s / **s**, which remain reserved exclusively for snap (see *Derivatives In 1D*) — no disambiguation is required when both appear in the same context, since the two symbols no longer collide.

10.8 Position And Trajectory In 3D

In three dimensions, position is represented by a vector:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

The three scalar component laws are treated with the same notation already defined for 1D.

This means that a 3D polynomial trajectory can be understood as three coordinated scalar polynomial laws:

$$x(t), \qquad y(t), \qquad z(t)$$

or, compactly:

$$\mathbf{p}(t) = [x(t),\ y(t),\ z(t)]^T$$

10.9 Derivatives In 3D

The derivative hierarchy extends component-wise:

$$\dot{\mathbf{p}}(t), \qquad \ddot{\mathbf{p}}(t), \qquad \mathbf{p}^{(3)}(t), \qquad \mathbf{p}^{(4)}(t)$$

The compact names are:

- velocity vector: $\mathbf{v}(t) = \dot{\mathbf{p}}(t)$
- acceleration vector: $\mathbf{a}(t) = \ddot{\mathbf{p}}(t)$
- jerk vector: $\mathbf{j}(t) = \mathbf{p}^{(3)}(t)$
- snap vector: $\mathbf{q}(t) = \mathbf{p}^{(4)}(t)$

The symbol $\mathbf{q}(t)$ is reserved here for vector snap only to avoid overloading the scalar letter s. If this creates confusion with quaternion notation in a future document, the explicit form $\mathbf{p}^{(4)}(t)$ should be preferred.

10.10 Boundary Conditions In 3D

The endpoint notation extends directly from the 1D case:

$$\mathbf{p}_0,\ \mathbf{p}_f,\ \mathbf{v}_0,\ \mathbf{v}_f,\ \mathbf{a}_0,\ \mathbf{a}_f,\ \mathbf{j}_0,\ \mathbf{j}_f,\ \mathbf{p}_0^{(4)},\ \mathbf{p}_f^{(4)}$$

Each vector quantity contains three scalar components. For example:

$$\mathbf{v}_0 = \begin{bmatrix} v_{x,0} \\ v_{y,0} \\ v_{z,0} \end{bmatrix}$$

This reinforces the main project idea: 3D treatment is built from the 1D theory, then extended by vector composition.

10.11 Segments, Multi-Segment Trajectories, And Blending

For a single segment, the trajectory law may be written as:

$$x_k(t)$$

in 1D, or:

$$\mathbf{p}_k(t)$$

in 3D, where k is the segment index.

A complete multi-segment trajectory is represented as an ordered set:

$$\mathcal{T} = \{\text{segment}_1, \text{segment}_2, \dots, \text{segment}_N\}$$

When blending is introduced between consecutive segments, the project uses the following notation:

- $b_{k\to k+1}^{start}$: point where blending toward the next segment starts;
- $b_{k\to k+1}^{end}$: point where blending from the previous segment ends.

If the context is explicitly geometric, those symbols may be written as positions:

$$x_{b,k}^{start}, \qquad x_{b,k}^{end}$$

in 1D, or:

$$\mathbf{p}_{b,k}^{start}, \qquad \mathbf{p}_{b,k}^{end}$$

in 3D.

10.12 Constraint Notation

The project uses generic constraint symbols first, then specific ones when needed.

Generic notation:

$$C = \{\text{type}, \text{value}\}$$

Typical scalar limits in 1D are:

- v_{max}: maximum admissible velocity;
- a_{max}: maximum admissible acceleration;
- j_{max}: maximum admissible jerk;
- s_{max}: maximum admissible snap.

When a constant-velocity sub-segment is required, the notation is:

$$x_{cv}^{start}, \qquad x_{cv}^{end}, \qquad v_{cv}$$

in 1D, and:

$$\mathbf{p}_{cv}^{start}, \qquad \mathbf{p}_{cv}^{end}, \qquad v_{cv}$$

in 3D.

The scalar v_{cv} denotes the requested constant speed magnitude. If direction matters in 3D, the corresponding constant velocity vector should be written as:

$$\mathbf{v}_{cv}$$

10.13 Reserved Orientation Notation

Orientation is not developed here, but the following symbols are reserved so that future documents remain coherent:

- Euler angles: ϕ, θ, ψ
- rotation matrix: $\mathbf{R}$
- quaternion: $\mathbf{q}$ or q, depending on the chosen convention

Because $\mathbf{q}$ may also be used for snap in some literature, future orientation documents must explicitly declare the chosen convention before use. Until then:

- use $\mathbf{p}^{(4)}(t)$ when referring to vector snap in potentially ambiguous contexts;
- use $\mathbf{R}$, ϕ, θ, ψ for reserved orientation notation;
- avoid quaternion derivations for now.

10.14 Procedure For Later Documents

To keep the full library consistent, every later document should follow this practical sequence:

1. start from the notation defined here;
2. introduce only the symbols actually needed by that topic;
3. keep the 1D derivation as the base case;
4. extend to 3D by component-wise interpretation or vector form;
5. declare any new symbol locally if it is not already defined here.

This simple procedure is important for study: it avoids re-learning a different notation in every document.

10.15 Worked Symbolic Considerations

The notation can already be read in a symbolic way without numerical data.

Example 1, 1D endpoint problem:

$$x(t): \quad (x_0, v_0, a_0, j_0, s_0) \to (x_f, v_f, a_f, j_f, s_f)$$

Example 2, 3D endpoint problem:

$$\mathbf{p}(t): \quad (\mathbf{p}_0, \mathbf{v}_0, \mathbf{a}_0) \to (\mathbf{p}_f, \mathbf{v}_f, \mathbf{a}_f)$$

Example 3, normalized-time formulation:

$$x(\tau) = \sum_{i=0}^{n} \alpha_i \tau^i, \qquad \tau \in [0, 1]$$

where α_i are the coefficients expressed in normalized time.

These examples are intentionally symbolic because the project prioritizes structural understanding before numerical substitution.

10.16 Open Questions

- **Resolved in `BoundaryConditions`**: for each polynomial degree, the minimal boundary-condition set is established there (necessary / possible / redundant classification, with a summary table for degrees 3, 5, 7, 9).
- **Resolved in `TrajectoryConstraints`**: whether 3D constraints should be imposed component-wise or on vector magnitude. This question also recurs in `NinthDegreePolynomialTheory`; it is intentionally not decided here, in isolation, but once the max/min velocity/acceleration/jerk/snap analysis (Req. 7–10) is developed in that document, with enough technical grounding to decide properly.
- **Open — to resolve during future orientation work** (see `04-future-topics.md`): should quaternion notation use q or **q** once snap notation is already present? No urgency until orientation is actually developed.
- **Resolved**: the polynomial coefficients collided with the acceleration symbol (a_i vs. $a(t), a_0, a_f$) in `NinthDegreePolynomialTheory`, `BoundaryConditions`, and `PolynomialCoefficientDetermination`. Coefficients are now denoted c_i throughout the library; see *Coefficient Notation Vs. Acceleration Notation* above.

10.17 Related Documents

- `06-theory-library-roadmap.md`
- `NinthDegreePolynomialTheory`
- `BoundaryConditions`
- `PolynomialCoefficientDetermination`

11 Ninth-Degree Polynomial Theory

11.1 Intro And Scope

This document introduces the ninth-degree polynomial as the first complete reference case for the project. Its role is twofold:

- to define the polynomial in forms that are useful for derivation and implementation;
- to study the derivative chain from position up to the sixth derivative, so the later trajectory-planning documents can refer to a shared theoretical base.

The treatment starts in 1D, because that is the base case adopted by the project. The 3D case is then obtained by applying the same scalar law independently to the three Cartesian components.

This document focuses on Requirement 2 of `01-requirements.md`:

- canonical form and Horner form in the time domain;
- derivatives up to the sixth derivative;
- discussion of the usefulness of using more or fewer derivatives in trajectory planning.

11.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: motion duration;
- $x(t)$: scalar position law in 1D;
- $c_0, c_1, \dots, c_9$: polynomial coefficients;
- $\mathcal{S}_0$, $\mathcal{S}_f$: initial and final state containers when endpoint data are grouped compactly;
- $v(t), a(t), j(t)$: velocity, acceleration, and jerk;
- $x^{(4)}(t), x^{(5)}(t), x^{(6)}(t)$: snap, crackle, and pop.

When normalized time is briefly mentioned, the symbol τ is used with $\tau \in [0, 1]$.

11.3 Why Degree Nine Is A Natural First Case

A ninth-degree polynomial has ten coefficients. This makes it especially useful as a first full study case because a trajectory segment can be determined by ten scalar conditions if the formulation requires them.

In the standard symmetric endpoint formulation, those ten conditions are naturally obtained by prescribing:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f,\ s_0,\ s_f$$

where s denotes snap.

This does not mean that every problem must always be posed this way. It means that degree nine is the first degree in this project that naturally accommodates endpoint information up to snap at both ends of a segment.

Using the state notation introduced in `SymbolsAndNomenclature.md`, the same endpoint specification may also be grouped as:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

This compact notation is useful when the full endpoint information is treated as a single conceptual object rather than as a list of separate scalar conditions.

11.4 Canonical Form In The Time Domain

The canonical form is the direct polynomial expansion in powers of time:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 + c_6 t^6 + c_7 t^7 + c_8 t^8 + c_9 t^9$$

The same expression can be written compactly as:

$$x(t) = \sum_{i=0}^{9} c_i t^i$$

This form is the most transparent one for theoretical work because every coefficient is visibly attached to its corresponding power of time.

11.4.1 Reading The Canonical Form

The canonical form is useful to read the structure of the motion law step by step:

1. the constant term c_0 is the value present even at $t = 0$;
2. the linear term $c_1 t$ contributes directly to initial slope;
3. higher powers progressively shape curvature and higher-order changes;
4. the highest-degree term $c_9 t^9$ gives the polynomial its ninth-degree character.

This reading is qualitative, but it is educationally useful because it connects algebraic structure to motion behavior.

11.5 Horner Form In The Time Domain

The same polynomial can be rewritten in nested form:

$$x(t) = c_0 + t\Big(c_1 + t(c_2 + t(c_3 + t(c_4 + t(c_5 + t(c_6 + t(c_7 + t(c_8 + tc_9)))))))\Big)$$

Equivalent right-nested notation is:

$$x(t) = (((((((((c_9 t + c_8)t + c_7)t + c_6)t + c_5)t + c_4)t + c_3)t + c_2)t + c_1)t + c_0)$$

These two expressions are algebraically identical. The difference is only in how the polynomial is organized for reading and evaluation.

11.5.1 How Horner Form Is Built

Starting from the canonical form:

$$x(t) = c_0 + c_1 t + c_2 t^2 + \cdots + c_9 t^9$$

factor t progressively:

$$x(t) = c_0 + t\left(c_1 + c_2 t + c_3 t^2 + \cdots + c_9 t^8\right)$$

then factor t again inside the parentheses:

$$x(t) = c_0 + t\left(c_1 + t\left(c_2 + c_3 t + c_4 t^2 + \cdots + c_9 t^7\right)\right)$$

and continue until the fully nested structure is obtained.

This is not a new polynomial. It is the same polynomial rewritten so that each step uses one multiplication by t and one addition.

11.6 Canonical Form Vs Horner Form

The two forms serve different purposes.

11.6.1 Canonical Form: Advantages

- It shows the polynomial degree immediately.
- It makes symbolic differentiation straightforward.
- It is convenient when setting up coefficient-determination equations.
- It is easier to inspect manually during theoretical study.
- It exposes clearly which coefficient multiplies which power of time.

11.6.2 Canonical Form: Limitations

- It is less efficient for repeated numerical evaluation.
- A direct implementation may compute many powers of t explicitly.
- Large powers of time can make numerical scaling less convenient.
- It is less natural for recursive evaluation.

11.6.3 Horner Form: Advantages

- It is computationally efficient for numerical evaluation.
- It avoids explicit computation of powers such as t^7, t^8, and t^9.
- It reduces the number of multiplications needed.
- It is often better suited for implementation in software.
- It is easy to evaluate repeatedly for many time samples.

11.6.4 Horner Form: Limitations

- It hides the power-by-power structure of the polynomial.
- It is less intuitive when deriving equations by hand.
- It is less transparent for teaching coefficient meaning.
- It is not the most convenient form for symbolic derivations.

11.6.5 Practical Interpretation

For this project, the best attitude is not to treat the two forms as competitors, but as complementary tools:

- canonical form is the preferred form for theory, derivation, and boundary-condition reasoning;
- Horner form is the preferred form for efficient numerical evaluation in software.

This is an important educational point: different mathematical forms of the same object become useful at different stages of the project.

11.7 General Derivative Rule

If

$$x(t) = \sum_{i=0}^{9} c_i t^i$$

then the k-th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^{9} \frac{i!}{(i-k)!} c_i t^{\,i-k}$$

This compact formula is useful because it shows the pattern behind all derivative expressions:

- every differentiation lowers the power of t by one;
- every differentiation multiplies the coefficient by the current exponent;
- terms of degree lower than k disappear in the k-th derivative.

11.8 Derivatives Of The Ninth-Degree Polynomial

Starting from

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 + c_6 t^6 + c_7 t^7 + c_8 t^8 + c_9 t^9$$

the derivatives up to the sixth are the following.

11.8.1 First Derivative: Velocity

$$v(t) = \dot{x}(t) = c_1 + 2c_2 t + 3c_3 t^2 + 4c_4 t^3 + 5c_5 t^4 + 6c_6 t^5 + 7c_7 t^6 + 8c_8 t^7 + 9c_9 t^8$$

Velocity is the first quantity that describes how position changes in time. It is also the most immediate kinematic limit in many planning problems.

11.8.2 Second Derivative: Acceleration

$$a(t) = \ddot{x}(t) = 2c_2 + 6c_3 t + 12c_4 t^2 + 20c_5 t^3 + 30c_6 t^4 + 42c_7 t^5 + 56c_8 t^6 + 72c_9 t^7$$

Acceleration measures how velocity changes. It is central whenever smooth dynamic evolution is more important than position alone.

11.8.3 Third Derivative: Jerk

$$j(t) = x^{(3)}(t) = 6c_3 + 24c_4 t + 60c_5 t^2 + 120c_6 t^3 + 210c_7 t^4 + 336c_8 t^5 + 504c_9 t^6$$

Jerk is often introduced when acceleration continuity is not sufficient and sharper changes need to be controlled.

11.8.4 Fourth Derivative: Snap

$$x^{(4)}(t) = 24c_4 + 120c_5 t + 360c_6 t^2 + 840c_7 t^3 + 1680c_8 t^4 + 3024c_9 t^5$$

Snap is especially relevant in a ninth-degree formulation because endpoint snap values can be included naturally among the ten scalar conditions.

11.8.5 Fifth Derivative: Crackle

$$x^{(5)}(t) = 120c_5 + 720c_6 t + 2520c_7 t^2 + 6720c_8 t^3 + 15120c_9 t^4$$

Crackle is less commonly used as a physical planning quantity, but it remains analytically useful because it is the derivative of snap.

11.8.6 Sixth Derivative: Pop

$$x^{(6)}(t) = 720c_6 + 5040c_7 t + 20160c_8 t^2 + 60480c_9 t^3$$

Pop is the derivative of crackle. It is rarely a primary planning constraint, but it is part of the natural derivative chain of a ninth-degree polynomial and can help understand higher-order variation.

11.9 What Changes In 3D

The 3D case does not introduce a new scalar theory. It applies the same ninth-degree law to each Cartesian component:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

with

$$x(t) = \sum_{i=0}^{9} c_i^{(x)} t^i, \qquad y(t) = \sum_{i=0}^{9} c_i^{(y)} t^i, \qquad z(t) = \sum_{i=0}^{9} c_i^{(z)} t^i$$

The same derivative chain applies component-wise:

$$\dot{\mathbf{p}}(t), \qquad \ddot{\mathbf{p}}(t), \qquad \mathbf{p}^{(3)}(t), \qquad \mathbf{p}^{(4)}(t), \qquad \mathbf{p}^{(5)}(t), \qquad \mathbf{p}^{(6)}(t)$$

So, from the viewpoint of this document:

- the 1D theory is the real mathematical core;
- the 3D extension is obtained by repeating that theory on three components;
- extra difficulty in 3D comes later from coupling choices, shared duration, and constraint interpretation, not from a different polynomial basis.

11.10 Why Fewer Derivatives May Be Enough

In many trajectory-planning problems, using fewer derivatives is entirely reasonable.

11.10.1 Position And Velocity Only

If the problem only requires reaching a target position with prescribed endpoint velocities, a lower-order description may be sufficient. This is useful when:

- the motion is simple;
- dynamic smoothness beyond velocity is not critical;
- the polynomial degree can be kept lower.

The advantage is simplicity. The limitation is that acceleration and higher-order behavior are left less controlled.

11.10.2 Up To Acceleration

Using position, velocity, and acceleration is common when smoother motion is needed. This usually gives a better compromise between control and complexity.

Acceleration-level reasoning is already very meaningful because:

- many mechanical effects are more closely related to acceleration than to position alone;
- acceleration continuity usually improves motion quality substantially;
- it avoids some abrupt changes that remain hidden if only velocity is considered.

11.10.3 Up To Jerk

Including jerk is often valuable when the transition smoothness itself becomes a design concern. Even without discussing actuators in detail, jerk remains educationally important because it captures how aggressively acceleration changes.

For this project, jerk is especially useful because it forms the bridge between basic trajectory geometry and genuinely smooth motion planning.

11.11 Why More Derivatives Can Be Useful

Studying derivatives beyond jerk is not always necessary, but it can still be useful for at least three reasons.

11.11.1 First Reason: Endpoint Specification Richness

Degree nine allows endpoint conditions up to snap at both ends. If snap is part of the formulation, then studying snap explicitly is not optional; it belongs to the definition of the segment itself.

11.11.2 Second Reason: Higher-Order Smoothness

Even if crackle and pop are not imposed as boundary conditions, they still describe how higher-order quantities evolve. This is useful when the goal is not only to obtain a valid trajectory, but to understand how smooth it really is.

11.11.3 Third Reason: Analytical Support For Later Constraints

Higher derivatives appear naturally when later documents analyze extrema:

- extrema of velocity are related to acceleration;
- extrema of acceleration are related to jerk;
- extrema of jerk are related to snap;
- extrema of snap are related to crackle.

So even if crackle and pop are not used as physical constraints, they remain mathematically useful.

11.12 Are Crackle And Pop Physically Necessary

Usually, no.

For most practical planning formulations, crackle and pop are not primary quantities to prescribe directly. In this project they are mainly useful because:

- they complete the derivative hierarchy of the ninth-degree polynomial;
- they help interpret the behavior of snap and higher-order smoothness;
- they support later analytical reasoning about maxima, minima, and continuity.

In other words:

- position, velocity, acceleration, and jerk are often directly meaningful;
- snap may become directly meaningful in high-smoothness formulations;
- crackle and pop are often more useful as analytical companions than as primary design targets.

11.13 The Special Role Of Snap In Degree Nine

Among the higher derivatives, snap has a special status in this project.

The reason is structural: a ninth-degree polynomial has ten coefficients, and the symmetric endpoint data set

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = \left(x_f, v_f, a_f, j_f, s_f\right)$$

provides exactly ten scalar conditions.

This makes snap the highest derivative that fits naturally into the standard two-endpoint determination of a ninth-degree segment.

That is why this document studies derivatives beyond jerk, but still treats snap as the highest derivative with a particularly strong structural role.

11.14 A Practical Evaluation Procedure

When the polynomial must be evaluated numerically at many time instants, the following workflow is useful:

1. derive the equations in canonical form;
2. determine the coefficients from the chosen boundary conditions;
3. convert the polynomial to Horner form for repeated evaluation;
4. sample the motion over time to obtain position and derivative profiles.

This workflow matches the educational goal of the project:

- theory remains readable in canonical form;
- implementation remains efficient in Horner form.

11.15 Worked Symbolic Considerations

11.15.1 Example 1: Full Ninth-Degree Segment In 1D

Consider the symbolic segment

$$x(t) = \sum_{i=0}^{9} c_i t^i$$

with endpoint data prescribed up to snap at both ends.

The key observation is not yet the solution for the coefficients, but the structural match:

- unknowns: $c_0, c_1, \dots, c_9$;
- scalar equations: initial and final values of position, velocity, acceleration, jerk, and snap.

This is why the ninth-degree polynomial is a natural theoretical anchor for the rest of the project.

11.15.2 Example 2: Why Higher Derivatives Matter For Extrema

Suppose a later document needs the maximum snap value. Then the stationary points of snap are found from:

$$\frac{d}{dt}\left(x^{(4)}(t)\right) = x^{(5)}(t) = 0$$

So crackle appears naturally even if it was never chosen as a planning constraint.

11.15.3 Example 3: 3D Interpretation

If

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

then a ninth-degree 3D segment is simply three synchronized ninth-degree scalar segments. The scalar theory does not change; only the bookkeeping grows.

11.16 Open Questions

- **Open — to resolve during the software design/implementation phase** (`src/`): should derivative profiles be evaluated directly from canonical formulas or from Horner-style recursive schemes? This is not a theory-library decision — both forms are proven equivalent above; the choice depends on measured performance in the actual C#/WPF implementation, not on the underlying math.
- **Resolved in `TrajectoryConstraints`**: whether limits should be interpreted component-wise or on vector magnitude when extending from 1D to 3D. This duplicates the same open question in `SymbolsAndNomenclature` — see the consolidated note there; it will be resolved once developed in `TrajectoryConstraints`, not decided here.
- **Open — to resolve during the application/UI design phase** (Req. 15): in the final application, which derivative levels should be visible by default in the charts, and which should remain optional?

11.17 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `BoundaryConditions`
- `PolynomialCoefficientDetermination`
- `TrajectoryConstraints`

12 Boundary Conditions

12.1 Intro And Scope

This document explains what boundary conditions are in the context of polynomial trajectory planning and lists the boundary conditions relevant to each polynomial degree currently in scope for the project: degree 3, 5, 7, and 9.

The goal of the document is educational before technical. It clarifies:

- what a boundary condition actually is;
- why the number of boundary conditions depends on the polynomial degree;
- how to distinguish necessary, possible, and redundant conditions;
- how the 1D reasoning extends to 3D.

This document covers the learning part of Requirement 3 in `01-requirements.md`.

12.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: motion duration;
- $x(t)$: scalar position law in 1D;
- $\mathcal{S}_0$, $\mathcal{S}_f$: initial and final state containers when endpoint data are grouped compactly;
- x_0, x_f: initial and final position;
- v_0, v_f: initial and final velocity;
- a_0, a_f: initial and final acceleration;
- j_0, j_f: initial and final jerk;
- s_0, s_f: initial and final snap.

When needed, the generic k-th derivative is written as $x^{(k)}(t)$.

12.3 What A Boundary Condition Is

A boundary condition is a prescribed value imposed on the trajectory or on one of its derivatives at a specific time, usually at the beginning or at the end of a segment.

Typical examples are:

$$x(t_0) = x_0, \qquad x(t_f) = x_f$$

for position, or:

$$\dot{x}(t_0) = v_0, \qquad \dot{x}(t_f) = v_f$$

for velocity.

So, in simple terms:

- a polynomial provides unknown coefficients;
- boundary conditions provide equations;
- when the number of independent equations matches the number of unknown coefficients, the polynomial can be determined uniquely.

This is the key idea behind the whole document.

12.4 Why The Number Of Boundary Conditions Depends On Degree

A polynomial of degree n has $n + 1$ coefficients:

$$x(t) = c_0 + c_1 t + c_2 t^2 + \cdots + c_n t^n$$

The unknowns are therefore:

$$c_0, c_1, c_2, \dots, c_n$$

which are $n + 1$ scalar unknowns.

If the duration T is already known and fixed, then the standard coefficient-determination problem requires:

$$n+1$$

independent scalar boundary conditions.

This immediately gives the count for the degrees currently in scope:

- degree 3: 4 conditions;
- degree 5: 6 conditions;
- degree 7: 8 conditions;
- degree 9: 10 conditions.

12.5 The Natural Symmetric Endpoint Pattern

For the odd degrees used in this project, there is a natural symmetric pattern based on matching the same derivative orders at the start and at the end of the segment.

The pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This pattern is natural because it gives exactly the required number of scalar conditions:

$$2 \times \frac{n+1}{2} = n+1$$

for odd degree n.

This is not the only possible formulation, but it is the most direct and educational one for this project.

12.6 Three Condition Classes

To keep the terminology consistent across the project, boundary conditions are divided into three classes.

12.6.1 Necessary Conditions

Necessary conditions are the conditions that must be specified in the chosen formulation to determine the polynomial uniquely.

In this document, the default formulation is:

- fixed polynomial degree;
- fixed start and end times;
- endpoint conditions only;
- symmetric use of derivative orders at the two endpoints.

Under this formulation, the necessary conditions are the ones listed degree by degree in the next sections.

12.6.2 Possible Conditions

Possible conditions are conditions that can be used in broader or alternative formulations, but are not required in the default formulation.

Examples:

- replacing one endpoint derivative condition with another independent condition;
- prescribing an interior-point condition;
- using higher-order derivative data while freeing another parameter such as duration;
- using a non-symmetric formulation between start and end conditions.

So “possible” does not mean “needed here”. It means “admissible in some extended formulation”.

12.6.3 Redundant Conditions

Redundant conditions are extra independent conditions beyond what the fixed-degree, fixed-duration polynomial can satisfy uniquely.

If a degree-n polynomial with fixed duration already has $n+1$ independent scalar conditions, then adding another independent condition makes the problem overconstrained.

In that situation, one of the following happens:

- the system has no exact solution;
- one of the conditions is not actually independent;
- the formulation must be changed, for example by increasing degree or adding free parameters.

This is why the distinction between possible and redundant is important:

- possible means usable in some reformulated problem;
- redundant means extra within the current fixed formulation.

12.7 Condition Catalogue In 1D

Before listing the degrees one by one, it is useful to define the basic catalogue of endpoint conditions in 1D:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f,\ s_0,\ s_f$$

These correspond to:

- position at start and end;
- velocity at start and end;
- acceleration at start and end;
- jerk at start and end;
- snap at start and end.

For the degrees currently in scope, this catalogue is sufficient for the main endpoint-based theory.

When a full endpoint condition set is treated as a compact object, the state notation from `SymbolsAndNomenclature.md` may also be used. For example, in the degree-9 case:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = \left(x_f, v_f, a_f, j_f, s_f\right)$$

This notation is particularly useful when discussing complete endpoint data sets without overloading the snap symbol s.

12.8 Degree 3

12.8.1 Polynomial Structure

A cubic polynomial has four coefficients:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$$

Therefore it needs four independent scalar conditions in the default formulation.

12.8.2 Necessary Conditions

The natural necessary set is:

$$x_0,\ x_f,\ v_0,\ v_f$$

This means:

$$x(t_0) = x_0, \qquad x(t_f) = x_f, \qquad \dot{x}(t_0) = v_0, \qquad \dot{x}(t_f) = v_f$$

12.8.3 Possible Conditions

Possible conditions in broader formulations may include:

- acceleration values at one or both endpoints;
- jerk values at one or both endpoints;
- an interior position value;
- unknown duration combined with a different choice of endpoint conditions.

These conditions are possible only if the full formulation is changed so that the total number of independent unknowns and equations remains balanced.

12.8.4 Redundant Conditions

In the default cubic formulation with fixed duration, the following types of additions are redundant:

- prescribing a_0 or a_f in addition to x_0, x_f, v_0, v_f;
- prescribing j_0 or j_f in addition to the same set;
- adding any extra independent endpoint condition beyond the original four.

So the basic educational message is simple: a cubic segment naturally controls position and velocity, not the full higher-order hierarchy.

12.9 Degree 5

12.9.1 Polynomial Structure

A quintic polynomial has six coefficients:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5$$

Therefore it needs six independent scalar conditions in the default formulation.

12.9.2 Necessary Conditions

The natural necessary set is:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f$$

This means:

$$x(t_0) = x_0, \qquad x(t_f) = x_f$$

$$\dot{x}(t_0) = v_0, \qquad \dot{x}(t_f) = v_f$$

$$\ddot{x}(t_0) = a_0, \qquad \ddot{x}(t_f) = a_f$$

12.9.3 Possible Conditions

Possible conditions in extended formulations may include:

- jerk at one or both endpoints;
- one or more interior conditions;
- duration treated as an unknown;
- a non-symmetric distribution of endpoint derivative conditions.

12.9.4 Redundant Conditions

In the default quintic formulation with fixed duration, the following become redundant if added independently:

- j_0, j_f;
- any snap endpoint condition;
- any extra independent condition beyond the six already listed.

The quintic therefore extends the cubic case by naturally controlling acceleration at the boundaries.

12.10 Degree 7

12.10.1 Polynomial Structure

A seventh-degree polynomial has eight coefficients:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 + c_6 t^6 + c_7 t^7$$

Therefore it needs eight independent scalar conditions in the default formulation.

12.10.2 Necessary Conditions

The natural necessary set is:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f$$

This means that position, velocity, acceleration, and jerk are prescribed at both endpoints.

12.10.3 Possible Conditions

Possible conditions in broader formulations may include:

- snap at one or both endpoints;
- interior constraints;
- alternative mixes of endpoint data;
- duration or other parameters promoted to unknowns.

12.10.4 Redundant Conditions

In the default seventh-degree formulation with fixed duration, the following become redundant if added independently:

- s_0, s_f;
- any extra endpoint condition beyond the eight already used;
- any other independent scalar condition that does not replace an existing one.

The educational interpretation is that the seventh-degree polynomial is the first degree in this project that naturally includes jerk at both ends.

12.11 Degree 9

12.11.1 Polynomial Structure

A ninth-degree polynomial has ten coefficients:

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 + c_6 t^6 + c_7 t^7 + c_8 t^8 + c_9 t^9$$

Therefore it needs ten independent scalar conditions in the default formulation.

12.11.2 Necessary Conditions

The natural necessary set is:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f,\ s_0,\ s_f$$

This means that position, velocity, acceleration, jerk, and snap are prescribed at both endpoints.

12.11.3 Possible Conditions

Possible conditions in broader formulations may include:

- crackle-related information if the problem is reformulated;
- interior-point constraints;
- asymmetric endpoint specifications;
- duration included among the unknowns;
- optimization-based or approximate fitting formulations.

12.11.4 Redundant Conditions

In the default ninth-degree formulation with fixed duration, the following become redundant if added independently:

- any additional crackle or pop endpoint condition;
- any extra interior scalar condition that is imposed without releasing another condition or adding unknowns;
- any extra independent scalar condition beyond the ten already used.

This is why snap has a special role in the degree-nine case: it is the highest derivative that still fits naturally into the standard symmetric endpoint formulation.

12.12 Summary Table For 1D

The degree-by-degree classification can be summarized as follows.

Degree	Coefficients	Necessary conditions in the default formulation	Examples of possible extra conditions in broader formulations	Redundant in the default fixed formulation
3	4	x_0, x_f, v_0, v_f	a_0, a_f, interior position, unknown duration	any fifth independent condition
5	6	$x_0, x_f, v_0, v_f, a_0, a_f$	j_0, j_f, interior conditions, unknown duration	any seventh independent condition
7	8	$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f$	s_0, s_f, interior conditions, unknown duration	any ninth independent condition
9	10	$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$	crackle-related data, interior conditions, unknown duration	any eleventh independent condition

This table should be read carefully:

- the “necessary” column refers only to the standard formulation adopted as default in this project;
- the “possible” column does not mean those conditions are added on top unchanged;
- the “redundant” column means “redundant unless the formulation itself is changed”.

12.13 Independence Matters As Much As Count

Counting conditions is necessary, but not sufficient. The conditions must also be independent.

For example, a degree-5 polynomial needs six scalar conditions, but not every group of six conditions is equally well chosen. If two conditions are linearly dependent in the coefficient system, they do not provide six independent equations.

So the complete rule is:

- correct number of conditions;
- independence of those conditions;
- consistency with the chosen formulation.

This point will become especially important in `PolynomialCoefficientDetermination`.

12.14 What Changes In 3D

The logic does not change in 3D. What changes is that the scalar 1D problem is applied to three Cartesian components.

If the three coordinates are treated independently, then each axis has its own set of scalar boundary conditions:

$$x_0,\ x_f,\ v_{x,0},\ v_{x,f},\ \dots$$

$$y_0,\ y_f,\ v_{y,0},\ v_{y,f},\ \dots$$

$$z_0,\ z_f,\ v_{z,0},\ v_{z,f},\ \dots$$

In compact vector notation, the corresponding endpoint conditions are:

$$\mathbf{p}_0,\ \mathbf{p}_f,\ \mathbf{v}_0,\ \mathbf{v}_f,\ \mathbf{a}_0,\ \mathbf{a}_f,\ \mathbf{j}_0,\ \mathbf{j}_f,\ \mathbf{p}_0^{(4)},\ \mathbf{p}_f^{(4)}$$

The educational idea is still the same:

- first understand the scalar case;
- then apply the same rule component-wise in 3D.

The main difference in 3D is therefore not the nature of the boundary conditions, but the bookkeeping and the later interpretation of multi-axis coordination.

12.15 Relation To The Data Model

`02-data-model.md` now defines `BoundaryConditions` as:

$$\text{BoundaryConditions} = \{\text{necessaryConditions}[], \text{possibleConditions}[], \text{redundantConditions}[]\}$$

This is the degree-parametrized, classified structure that this document's necessary/possible/redundant classification calls for — the revision has been carried out, replacing the earlier fixed set of ten scalar fields (`startPosition/endPosition`, etc.) that matched only the degree-9 default formulation.

The three arrays hold:

- `necessaryConditions[]`: the conditions required by the chosen degree and formulation;
- `possibleConditions[]`: conditions usable in alternative formulations;
- `redundantConditions[]`: conditions that exceed the default fixed formulation.

What remains open is not *whether* to revise the structure (done), but the internal representation of each individual condition within those arrays — see the corresponding open question below, to be resolved together with `PolynomialCoefficientDetermination`.

In later implementation, each condition will likely need a more explicit internal representation, for example:

- derivative order;
- endpoint identifier;
- symbolic or numeric value.

12.16 Worked Symbolic Considerations

12.16.1 Example 1: Cubic Segment

For a cubic segment with fixed duration, the unknown coefficients are:

$$c_0,\ c_1,\ c_2,\ c_3$$

The natural endpoint conditions are:

$$x_0,\ x_f,\ v_0,\ v_f$$

This gives four independent scalar equations for four unknowns.

If one now adds a_0 and a_f as prescribed endpoint accelerations without changing the formulation, the cubic problem becomes overconstrained.

12.16.2 Example 2: Quintic Segment

For a quintic segment, the unknown coefficients are:

$$c_0,\ c_1,\ c_2,\ c_3,\ c_4,\ c_5$$

The natural endpoint conditions are:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f$$

This is why quintic polynomials are often associated with endpoint control up to acceleration.

12.16.3 Example 3: Ninth-Degree Segment

For a ninth-degree segment, the natural symmetric endpoint set is:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

This gives ten scalar conditions, matching the ten coefficients of the polynomial exactly.

That exact structural match is one of the main reasons why the degree-nine case is the central reference of the project.

12.17 Open Questions

- **Resolved**: whether `02-data-model.md`'s `BoundaryConditions` structure should be revised to the degree-parametrized, classified form — it has been, see *Relation To The Data Model*. **Still open, deferred to** `PolynomialCoefficientDete` the internal representation of each individual condition within `necessaryConditions[]` / `possibleConditions[]` / `redundantConditions[]` (by derivative order vs. a more descriptive enum-based type) — this is `PolynomialCoefficientDeterm` own Open Question on the same topic, not yet decided there either.
- **Open — to resolve if/when alternative formulations are introduced** (interior points, optimization-based fitting): should the project keep the same necessary/possible/redundant naming, or add a fourth category for "active in the current variant"? No urgency yet, since the project still uses the fixed default formulation.
- **Open — to resolve during the application/UI design phase** (Req. 15): for 3D trajectories, should the user interface expose boundary conditions axis by axis, or primarily in vector form?

12.18 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `NinthDegreePolynomialTheory.md`
- `PolynomialCoefficientDetermination`
- `02-data-model.md`

13 Polynomial Coefficient Determination

13.1 Intro And Scope

This document explains how to determine the coefficients of a polynomial trajectory once the necessary boundary data has been specified.

It covers two connected themes:

- the technical procedure required by Requirement 3 in `01-requirements.md`;
- the learning-oriented comparison between absolute time and normalized time required by Requirement 4.

The discussion starts in 1D, because that is the conceptual base of the project. The 3D case is then obtained by applying the same scalar procedure to the Cartesian components.

The focus is on degrees 3, 5, 7, and 9, which are the degrees currently in scope for the project.

13.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: segment duration;
- $\tau = \frac{t - t_0}{T}$: normalized time;
- $x(t)$: scalar position law in 1D written in absolute time;
- $x(\tau)$: scalar position law in normalized time;
- n: polynomial degree;
- c_i: coefficients of the polynomial when written in absolute time;
- α_i: coefficients of the polynomial when written in normalized time;
- $\mathcal{S}_0$, $\mathcal{S}_f$: endpoint state containers when compact notation is useful.

When derivatives are written in the normalized variable, the notation

$$\frac{d^k x}{d\tau^k}$$

is preferred in order to keep it clearly distinct from derivatives with respect to absolute time.

13.3 Problem Statement

The coefficient-determination problem can be stated as follows.

Given:

- a polynomial degree n;
- a fixed segment duration T;
- the necessary boundary conditions for that degree;

determine the polynomial coefficients so that all required endpoint conditions are satisfied exactly.

For the degrees currently used in the project, the default endpoint pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This means that coefficient determination is always a square linear problem in the default formulation:

- degree 3: 4 unknown coefficients, 4 scalar equations;
- degree 5: 6 unknown coefficients, 6 scalar equations;
- degree 7: 8 unknown coefficients, 8 scalar equations;
- degree 9: 10 unknown coefficients, 10 scalar equations.

13.4 Core Idea

A polynomial trajectory has unknown coefficients. Each boundary condition produces one scalar equation in those coefficients.

So the practical workflow is always:

1. choose the polynomial degree;
2. write the polynomial and its derivatives;
3. evaluate the required derivatives at the endpoints;
4. assemble the linear system;
5. solve for the coefficients;
6. verify the result by substitution.

The mathematical heart of the method is therefore not a special trick, but a structured linear system.

13.5 Absolute-Time Formulation

13.5.1 General Polynomial

In absolute time, the polynomial is written as:

$$x(t) = \sum_{i=0}^{n} c_i t^i$$

with unknown coefficient vector:

$$\mathbf{c} = \begin{bmatrix} c_0 & c_1 & \dots & c_n \end{bmatrix}^T$$

13.5.2 General Derivative Rule

The k-th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^{n} \frac{i!}{(i-k)!} c_i t^{\,i-k}$$

This formula is enough to generate every row of the coefficient-determination system.

13.5.3 Derivative Row Operators

For compact notation, define the row vector $\mathbf{D}_k(t)$ as the coefficient row that maps $\mathbf{c}$ to the k-th derivative value at time t:

$$x^{(k)}(t) = \mathbf{D}_k(t)\,\mathbf{c}$$

For degree 9, the first five rows are:

$$\mathbf{D}_0(t) = \begin{bmatrix} 1 & t & t^2 & t^3 & t^4 & t^5 & t^6 & t^7 & t^8 & t^9 \end{bmatrix}$$

$$\mathbf{D}_1(t) = \begin{bmatrix} 0 & 1 & 2t & 3t^2 & 4t^3 & 5t^4 & 6t^5 & 7t^6 & 8t^7 & 9t^8 \end{bmatrix}$$

$$\mathbf{D}_2(t) = \begin{bmatrix} 0 & 0 & 2 & 6t & 12t^2 & 20t^3 & 30t^4 & 42t^5 & 56t^6 & 72t^7 \end{bmatrix}$$

$$\mathbf{D}_3(t) = \begin{bmatrix} 0 & 0 & 0 & 6 & 24t & 60t^2 & 120t^3 & 210t^4 & 336t^5 & 504t^6 \end{bmatrix}$$

$$\mathbf{D}_4(t) = \begin{bmatrix} 0 & 0 & 0 & 0 & 24 & 120t & 360t^2 & 840t^3 & 1680t^4 & 3024t^5 \end{bmatrix}$$

These rows are directly derived from `NinthDegreePolynomialTheory.md`.

13.6 Degree-Specific Linear Systems In Absolute Time

The default square systems for the degrees in scope are obtained by stacking the derivative rows at t_0 and t_f.

13.6.1 Degree 3

For

$$x(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$$

use:

$$\mathbf{c}_3 = \begin{bmatrix} c_0 & c_1 & c_2 & c_3 \end{bmatrix}^T$$

and:

$$\mathbf{b}_3 = \begin{bmatrix} x_0 & v_0 & x_f & v_f \end{bmatrix}^T$$

The system is:

$$\mathbf{A}_3^{abs}\, \mathbf{c}_3 = \mathbf{b}_3$$

with:

$$\mathbf{A}_3^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \end{bmatrix}$$

where each row is truncated to degree 3.

13.6.2 Degree 5

Use:

$$\mathbf{b}_5 = \begin{bmatrix} x_0 & v_0 & a_0 & x_f & v_f & a_f \end{bmatrix}^T$$

and:

$$\mathbf{A}_5^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \end{bmatrix}$$

again truncated to degree 5.

13.6.3 Degree 7

Use:

$$\mathbf{b}_7 = \begin{bmatrix} x_0 & v_0 & a_0 & j_0 & x_f & v_f & a_f & j_f \end{bmatrix}^T$$

and:

$$\mathbf{A}_7^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \end{bmatrix}$$

truncated to degree 7.

13.6.4 Degree 9

Use the compact state notation:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

or, equivalently, the endpoint vector:

$$\mathbf{b}_9 = \begin{bmatrix} x_0 & v_0 & a_0 & j_0 & s_0 & x_f & v_f & a_f & j_f & s_f \end{bmatrix}^T$$

Then:

$$\mathbf{A}_9^{abs}\, \mathbf{c}_9 = \mathbf{b}_9$$

with:

$$\mathbf{A}_9^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_4(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \\ \mathbf{D}_4(t_f) \end{bmatrix}$$

This is the direct absolute-time coefficient-determination system for the degree-9 case.

13.7 Step-By-Step Procedure In Absolute Time

The practical procedure in absolute time is the same for every supported degree.

1. Choose the degree n and identify the necessary endpoint conditions from `BoundaryConditions.md`.
2. Write the polynomial $x(t)$ in canonical form with unknown coefficients.
3. Compute the derivatives needed by that degree.
4. Evaluate those derivatives at t_0 and t_f.
5. Assemble the matrix $\mathbf{A}^{abs}$ and the right-hand side vector $\mathbf{b}$.

6. Solve the linear system for the coefficient vector **c**.
7. Substitute the solution back into the polynomial and verify all endpoint conditions.

Conceptually, this is simple. In practice, the algebra becomes longer as the degree increases.

13.8 Why Normalized Time Is Worth Introducing

Absolute time works directly, but it is not always the most convenient variable.

When the segment duration is fixed, it is often useful to define:

$$\tau = \frac{t - t_0}{T}, \qquad \tau \in [0, 1]$$

and write the same segment as:

$$x(\tau) = \sum_{i=0}^{n} \alpha_i \tau^i$$

The important idea is that:

- t belongs to the physical time axis;
- τ belongs to a normalized local segment axis;
- the two descriptions represent the same trajectory segment, but with different coefficients.

If $t_0 \neq 0$, the sets $\{c_i\}$ and $\{\alpha_i\}$ are not related by a simple one-to-one scaling. The normalized representation is centered on the local interval $[0, 1]$, whereas the absolute-time representation is written directly in powers of the global variable t.

13.9 Derivative Scaling Between t And τ

Because

$$\tau = \frac{t - t_0}{T}$$

we have:

$$\frac{d\tau}{dt} = \frac{1}{T}$$

Therefore:

$$\frac{dx}{dt} = \frac{dx}{d\tau}\frac{d\tau}{dt} = \frac{1}{T}\frac{dx}{d\tau}$$

and, more generally:

$$\frac{d^k x}{dt^k} = \frac{1}{T^k}\frac{d^k x}{d\tau^k}$$

Equivalently:

$$\frac{d^k x}{d\tau^k} = T^k \frac{d^k x}{dt^k}$$

This scaling rule is the key bridge between physical endpoint data and the normalized-time coefficient system.

13.10 Degree-9 System In Normalized Time

For degree 9, write:

$$x(\tau) = \alpha_0 + \alpha_1\tau + \alpha_2\tau^2 + \alpha_3\tau^3 + \alpha_4\tau^4 + \alpha_5\tau^5 + \alpha_6\tau^6 + \alpha_7\tau^7 + \alpha_8\tau^8 + \alpha_9\tau^9$$

with coefficient vector:

$$\alpha = \begin{bmatrix}\alpha_0 & \alpha_1 & \alpha_2 & \alpha_3 & \alpha_4 & \alpha_5 & \alpha_6 & \alpha_7 & \alpha_8 & \alpha_9\end{bmatrix}^T$$

At $\tau = 0$, the first five endpoint equations become:

$$x(0) = \alpha_0 = x_0$$

$$\frac{dx}{d\tau}(0) = \alpha_1 = Tv_0$$

$$\frac{d^2x}{d\tau^2}(0) = 2\alpha_2 = T^2a_0$$

$$\frac{d^3x}{d\tau^3}(0) = 6\alpha_3 = T^3j_0$$

$$\frac{d^4x}{d\tau^4}(0) = 24\alpha_4 = T^4s_0$$

At $\tau = 1$, the remaining equations are:

$$x(1) = \sum_{i=0}^{9}\alpha_i = x_f$$

$$\frac{dx}{d\tau}(1) = \alpha_1 + 2\alpha_2 + 3\alpha_3 + 4\alpha_4 + 5\alpha_5 + 6\alpha_6 + 7\alpha_7 + 8\alpha_8 + 9\alpha_9 = Tv_f$$

$$\frac{d^2x}{d\tau^2}(1) = 2\alpha_2 + 6\alpha_3 + 12\alpha_4 + 20\alpha_5 + 30\alpha_6 + 42\alpha_7 + 56\alpha_8 + 72\alpha_9 = T^2a_f$$

$$\frac{d^3x}{d\tau^3}(1) = 6\alpha_3 + 24\alpha_4 + 60\alpha_5 + 120\alpha_6 + 210\alpha_7 + 336\alpha_8 + 504\alpha_9 = T^3j_f$$

$$\frac{d^4x}{d\tau^4}(1) = 24\alpha_4 + 120\alpha_5 + 360\alpha_6 + 840\alpha_7 + 1680\alpha_8 + 3024\alpha_9 = T^4s_f$$

So the normalized system is:

$$\mathbf{A}_9^{norm}\,\alpha = \mathbf{b}_9^{norm}$$

with:

$$\mathbf{A}_9^{norm} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 24 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 0 & 0 & 2 & 6 & 12 & 20 & 30 & 42 & 56 & 72 \\ 0 & 0 & 0 & 6 & 24 & 60 & 120 & 210 & 336 & 504 \\ 0 & 0 & 0 & 0 & 24 & 120 & 360 & 840 & 1680 & 3024 \end{bmatrix}$$

and:

$$\mathbf{b}_9^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & T^3j_0 & T^4s_0 & x_f & Tv_f & T^2a_f & T^3j_f & T^4s_f \end{bmatrix}^T$$

This is one of the main practical advantages of normalized time: for a fixed degree, the matrix is constant and can be reused for every segment.

13.11 How The Lower Degrees Fit The Same Normalized Pattern

The same normalized-time logic applies to every supported degree.

13.11.1 Degree 3

Use:

$$\mathbf{b}_3^{norm} = \begin{bmatrix} x_0 & Tv_0 & x_f & Tv_f \end{bmatrix}^T$$

with a constant 4×4 normalized matrix.

13.11.2 Degree 5

Use:

$$\mathbf{b}_5^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & x_f & Tv_f & T^2a_f \end{bmatrix}^T$$

with a constant 6×6 normalized matrix.

13.11.3 Degree 7

Use:

$$\mathbf{b}_7^{norm} = \begin{bmatrix} x_0 & Tv_0 & T^2a_0 & T^3j_0 & x_f & Tv_f & T^2a_f & T^3j_f \end{bmatrix}^T$$

with a constant 8×8 normalized matrix.

13.11.4 Degree 9

Use the degree-9 system written in the previous section.

So the pattern is stable:

- the normalized matrix depends only on degree;
- the right-hand side contains the physical endpoint data scaled by powers of T.

13.12 Absolute Time Vs Normalized Time

The two formulations represent the same trajectory, but they are useful in different ways.

13.12.1 Absolute Time: Main Advantages

- It keeps the polynomial directly expressed in the physical variable t.
- It makes the final law immediately readable in the original time axis.
- It avoids introducing an extra variable.
- It is conceptually straightforward when $t_0 = 0$ and durations are simple.

13.12.2 Absolute Time: Main Limitations

- The system matrix changes with t_0 and t_f.
- Large or awkward time values generate large powers such as t_f^8 or t_f^9.
- The algebra becomes less uniform from one segment to another.
- Numerical conditioning can become worse when absolute times are large.

13.12.3 Normalized Time: Main Advantages

- The segment always lives on the same interval $[0, 1]$.
- For a given degree, the system matrix is constant and reusable.
- The formulation is usually cleaner for symbolic derivation.
- Numerical scaling is often better because the variable does not grow beyond the local interval.
- Multi-segment implementations become more uniform because every segment uses the same local coordinate.

13.12.4 Normalized Time: Main Limitations

- Physical derivatives must be rescaled by powers of T.
- The coefficients α_i are not the same as the absolute-time coefficients c_i.
- If a final expression explicitly in powers of t is required, an extra conversion step is needed.
- The method introduces one extra conceptual layer for beginners.

13.12.5 Practical Interpretation For This Project

For this project, the most balanced approach is:

- use normalized time as the main working variable for coefficient determination;
- keep absolute time as the interpretation layer in which physical durations and derivative values are specified;
- evaluate the final trajectory numerically either in normalized time or, if needed, after conversion to absolute time.

This keeps the derivation clean without losing contact with physical meaning.

13.13 Recommended Determination Strategy

The following strategy is recommended for the project.

1. Express the trajectory segment in normalized time.
2. Build the constant normalized matrix associated with the chosen degree.
3. Assemble the right-hand side from the physical endpoint data scaled by T^k.
4. Solve for the normalized coefficients α_i.
5. Evaluate the trajectory and its derivatives using τ as the local segment variable.
6. Convert to an absolute-time polynomial only if that representation is explicitly needed.

This recommendation is especially useful for software implementation because it separates:

- physical inputs;
- degree-dependent matrix structure;
- reusable numerical evaluation.

13.14 Independence And Solvability

Counting equations is not enough. The boundary conditions must also be independent, and the matrix must be invertible.

So a well-posed coefficient-determination problem requires:

- the correct number of scalar conditions for the chosen degree;

- independence of those conditions;
- a consistent time definition with $T \neq 0$.

If these conditions fail, the system may become:

- underdetermined, if too few independent conditions are supplied;
- overdetermined, if too many independent conditions are imposed;
- singular, if the conditions are formally counted correctly but not independent.

This connects directly to the necessary / possible / redundant classification introduced in `BoundaryConditions.md`.

13.15 Relation To The Data Model

`02-data-model.md` currently defines:

$$\text{Polynomial} = \{\text{degree}, \text{coefficients}, \text{boundaryConditions}\}$$

and:

$$\text{BoundaryConditions} = \{\text{necessaryConditions}[], \text{possibleConditions}[], \text{redundantConditions}[]\}$$

From the viewpoint of coefficient determination, this means:

- `degree` selects the size and structure of the determination system;
- `necessaryConditions[]` provides the data that actually feeds the square linear system;
- `possibleConditions[]` may become relevant only in alternative formulations;
- `redundantConditions[]` should not be injected into the default fixed-degree system unchanged.

So the coefficient-determination procedure is the first place where the theoretical classification of boundary conditions becomes operational.

A future implementation-oriented refinement will likely need each necessary condition to carry at least:

- derivative order;
- endpoint identifier;
- scalar or vector value;
- dimensional context (`1D` or `3D`).

13.16 What Changes In 3D

The determination logic does not change in 3D. The same scalar problem is solved component-wise.

For example, in a degree-9 segment:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

the project can determine:

- one coefficient vector for the x component;
- one coefficient vector for the y component;
- one coefficient vector for the z component.

If the duration T is shared by all three axes, then the same normalized matrix is reused three times with three different right-hand side vectors.

This is one of the strongest reasons for keeping the 1D theory as the base case: the 3D extension is structurally repetitive rather than conceptually new.

13.17 Worked Symbolic Considerations

13.17.1 Example 1: Degree 3

For a cubic segment with fixed duration, the unknown coefficients are:

$$c_0,\ c_1,\ c_2,\ c_3$$

The natural endpoint data are:

$$x_0,\ v_0,\ x_f,\ v_f$$

These four scalar conditions generate a 4×4 square system. If the system matrix is invertible, the four coefficients are determined uniquely.

13.17.2 Example 2: Degree 9 In Normalized Time

For the degree-9 case, the endpoint data may be grouped as:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0)\,, \qquad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

The normalized system then uses:

$$x_0,\ Tv_0,\ T^2a_0,\ T^3j_0,\ T^4s_0,\ x_f,\ Tv_f,\ T^2a_f,\ T^3j_f,\ T^4s_f$$

as right-hand side data. The matrix stays constant; only the scaled endpoint vector changes from segment to segment.

13.17.3 Example 3: Why Normalized Time Helps In Multi-Segment Work

Suppose two trajectory segments have the same polynomial degree but different durations. In absolute time, their matrices are different because their endpoint times are different.

In normalized time, both segments use the same coefficient matrix. Only the right-hand side changes through the powers of each segment duration:

- one segment uses T_1, T_1^2, T_1^3, and so on;
- the other uses T_2, T_2^2, T_2^3, and so on.

This is mathematically cleaner and implementation-friendly.

13.18 Open Questions

- **Open — to resolve during software implementation** (`src/`): should the application store only normalized coefficients internally, or also keep an absolute-time coefficient expansion for inspection/export?
- **Deferred to the later constraints/blending documents**: when a formulation moves beyond the default necessary-condition set, should the software solve a restructured exact system, or transition to constrained/optimization-based formulations?
- **Open — to resolve during API/data-structure design**: should each condition within `necessaryConditions[]` / `possibleConditions[]` / `redundantConditions[]` be stored as a generic condition record, or as degree-specific strongly typed structures for degrees 3, 5, 7, and 9? (The classified array structure itself is already in place in `02-data-model.md`; only the internal representation of each entry remains open. Same open question as in `BoundaryConditions.md` — not yet decided in either document.)

13.19 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `NinthDegreePolynomialTheory.md`
- `BoundaryConditions.md`
- `TrajectoryConstraints`
- `02-data-model.md`

14 Trajectory Constraints

14.1 Intro And Scope

This document studies trajectory constraints for the polynomial trajectories currently in scope, with the ninth-degree polynomial as the main technical reference case.

It covers the requirement cluster formed by items 7, 8, 9, and 10 in `01-requirements.md`:

- determination of maxima and minima for position, velocity, acceleration, jerk, and snap;
- procedure to impose a velocity limit;
- procedure to impose an acceleration limit;
- procedure to impose a jerk limit.

The treatment starts in 1D and then extends to 3D. In keeping with `03-assumptions.md`, the discussion remains theoretical, symbolic-first, and independent of actuator dynamics or robot kinematics.

14.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: segment duration;
- $\tau = \frac{t - t_0}{T}$: normalized time;
- $x(t)$: scalar position law in 1D;
- $v(t) = \dot{x}(t)$: velocity;
- $a(t) = \ddot{x}(t)$: acceleration;
- $j(t) = x^{(3)}(t)$: jerk;
- $s(t) = x^{(4)}(t)$: snap;
- $x^{(5)}(t)$: crackle;
- $\mathbf{p}(t)$, $\mathbf{v}(t)$, $\mathbf{a}(t)$, $\mathbf{j}(t)$, $\mathbf{p}^{(4)}(t)$: 3D position and derivative vectors;
- v_{max}: admissible velocity limit;
- a_{max}: admissible acceleration limit;
- j_{max}: admissible jerk limit.

Unless stated otherwise, all extrema are searched over the closed interval:

$$t \in [t_0, t_f]$$

or, equivalently:

$$\tau \in [0, 1]$$

14.3 Constraint Context

In this project, a constraint is a condition imposed on the trajectory after the polynomial structure has been defined.

The most important kinematic constraints considered here are:

- bounds on velocity;
- bounds on acceleration;
- bounds on jerk.

Snap is included in the extrema analysis because Requirement 7 asks for it explicitly, even though there is no separate limit-imposition requirement for snap at this stage.

Position is also included in the extrema analysis, not because it is normally a “limit” in the same sense as velocity or acceleration, but because understanding where position reaches local or global extrema is part of understanding the trajectory profile.

14.4 Core Principle: Candidate Points For Extrema

For any scalar function $f(t)$ on a closed interval, the global maximum and minimum can only occur at:

- the interval endpoints;
- interior stationary points where $\frac{df}{dt} = 0$;
- interior points where the function is not differentiable.

Polynomial trajectories and their derivatives are smooth, so the third item does not apply here.

Therefore, the general constraint-analysis rule in this project is:

1. identify the scalar quantity of interest;
2. differentiate it;
3. solve the stationarity equation inside the interval;
4. evaluate the original quantity at all candidate points;
5. compare those values to obtain maxima and minima.

This rule is simple, but it drives almost the whole document.

14.5 Why Higher Derivatives Matter For Extrema

The derivative chain studied in `NinthDegreePolynomialTheory.md` now becomes operational.

For a degree-9 polynomial:

- extrema of position depend on velocity;
- extrema of velocity depend on acceleration;
- extrema of acceleration depend on jerk;
- extrema of jerk depend on snap;
- extrema of snap depend on crackle.

So the higher derivatives are not only abstract quantities. They are exactly the tools needed to locate the candidate extrema of the lower-order profiles.

14.6 Position Extrema In 1D

14.6.1 General Rule

To find maxima and minima of position $x(t)$ over $[t_0, t_f]$, solve:

$$v(t) = \dot{x}(t) = 0$$

for all roots inside the interval, then evaluate:

- $x(t_0)$;
- $x(t_f)$;
- $x(t_i)$ for every interior root t_i of $v(t)$.

The largest of these values is the maximum position, and the smallest is the minimum position.

14.6.2 Degree-9 Specialization

For a degree-9 polynomial:

$$x(t) = \sum_{i=0}^{9} a_i t^i$$

the velocity is degree 8:

$$v(t) = \dot{x}(t)$$

So the position-extrema problem reduces to solving a degree-8 polynomial equation.

14.6.3 Important Interpretation

The second derivative test may help classify local extrema, but for global max/min over a finite interval it is not strictly necessary. The safest method is direct comparison of all candidate values.

14.7 Velocity Extrema In 1D

14.7.1 General Rule

To find maxima and minima of velocity $v(t)$ over $[t_0, t_f]$, solve:

$$a(t) = \ddot{x}(t) = 0$$

for all roots inside the interval, then evaluate:

- $v(t_0)$;
- $v(t_f)$;
- $v(t_i)$ for every interior root t_i of $a(t)$.

14.7.2 Degree-9 Specialization

For a degree-9 polynomial, acceleration is degree 7:

$$a(t) = \ddot{x}(t)$$

So the velocity-extrema problem reduces to solving a degree-7 polynomial equation.

14.7.3 Absolute Velocity

When a velocity limit is expressed as a magnitude bound:

$$|v(t)| \leq v_{max}$$

the relevant quantity is not the signed maximum alone, but:

$$\max_{t \in [t_0, t_f]} |v(t)|$$

This can be obtained by evaluating $|v(t)|$ at the same candidate points used for the signed extrema, or equivalently by checking both the maximum and minimum signed values.

14.8 Acceleration Extrema In 1D

14.8.1 General Rule

To find maxima and minima of acceleration $a(t)$, solve:

$$j(t) = x^{(3)}(t) = 0$$

inside the interval, then compare:

- $a(t_0)$;
- $a(t_f)$;
- $a(t_i)$ for each interior root t_i of $j(t)$.

14.8.2 Degree-9 Specialization

For the degree-9 polynomial, jerk is degree 6:

$$j(t) = x^{(3)}(t)$$

So acceleration-extrema analysis reduces to solving a degree-6 polynomial equation.

14.8.3 Absolute Acceleration

For an acceleration bound:

$$|a(t)| \leq a_{max}$$

the quantity to monitor is:

$$\max_{t \in [t_0, t_f]} |a(t)|$$

again obtained by evaluating the acceleration at all candidate points and taking absolute values.

14.9 Jerk Extrema In 1D

14.9.1 General Rule

To find maxima and minima of jerk $j(t)$, solve:

$$s(t) = x^{(4)}(t) = 0$$

inside the interval, then compare:

- $j(t_0)$;
- $j(t_f)$;
- $j(t_i)$ for each interior root t_i of $s(t)$.

14.9.2 Degree-9 Specialization

For the degree-9 polynomial, snap is degree 5:

$$s(t) = x^{(4)}(t)$$

So jerk-extrema analysis reduces to solving a degree-5 polynomial equation.

14.9.3 Absolute Jerk

For a jerk limit:

$$|j(t)| \leq j_{max}$$

the relevant quantity is:

$$\max_{t \in [t_0, t_f]} |j(t)|$$

computed from the same candidate-point set.

14.10 Snap Extrema In 1D

14.10.1 General Rule

To find maxima and minima of snap $s(t)$, solve:

$$x^{(5)}(t) = 0$$

inside the interval, then compare:

- $s(t_0)$;
- $s(t_f)$;
- $s(t_i)$ for each interior root t_i of $x^{(5)}(t)$.

14.10.2 Degree-9 Specialization

For the degree-9 polynomial, crackle is degree 4:

$$x^{(5)}(t)$$

So snap-extrema analysis reduces to solving a quartic equation. This is the highest-order extrema search in this document whose stationarity equation still falls inside the classical closed-form range.

14.11 Summary Of Stationarity Equations

The extrema-search structure for the degree-9 case can be summarized as:

Quantity analysed	Candidate interior points come from
position $x(t)$	roots of $v(t) = 0$
velocity $v(t)$	roots of $a(t) = 0$
acceleration $a(t)$	roots of $j(t) = 0$
jerk $j(t)$	roots of $s(t) = 0$
snap $s(t)$	roots of $x^{(5)}(t) = 0$

This table is operationally important: it tells which polynomial must be solved for each profile.

14.12 Symbolic Setup Vs Numerical Root Extraction

The project prefers symbolic treatment, but extrema detection introduces an unavoidable practical distinction:

- the polynomial expressions and the stationarity equations can be written symbolically;
- the actual extraction of all real roots in the interval may require numerical methods.

This happens because:

- quartic equations still admit closed-form symbolic solutions in principle;
- quintic and higher-degree polynomial equations do not, in general, admit a universal closed-form solution by radicals.

Therefore, for the degree-9 case:

- snap extrema can still be tied to a quartic stationarity equation;
- jerk, acceleration, velocity, and position extrema typically require numerical root extraction for the stationarity polynomial.

This is not a contradiction with the symbolic-first goal. The theory remains symbolic; the final root-finding step becomes numerical when algebra no longer admits a general closed form.

14.13 Generic 1D Extrema Algorithm

The following algorithm applies to any scalar quantity among:

$$x(t), \qquad v(t), \qquad a(t), \qquad j(t), \qquad s(t)$$

1. Select the target profile $f(t)$.
2. Compute its derivative $f'(t)$.
3. Solve $f'(t) = 0$ and retain only the real roots inside $[t_0, t_f]$.
4. Build the candidate set:

$$\{t_0,\ t_f,\ t_1,\ t_2,\ \dots,\ t_m\}$$

5. Evaluate $f(t)$ at every candidate point.
6. Compare those values to obtain:
   - global maximum;
   - global minimum;
   - if needed, maximum absolute value.

This is the baseline algorithm for Requirement 7.

14.14 Practical Root-Finding Note

For software implementation, the root-finding step should be understood as:

- formulate the stationarity polynomial exactly from the coefficients;
- compute all its roots numerically;
- discard complex roots;
- discard real roots lying outside the interval;
- keep the remaining roots as extrema candidates.

Possible numerical strategies include:

- companion-matrix eigenvalue extraction;
- robust polynomial root solvers;
- interval-based refinement after coarse root localization.

The choice of numerical solver belongs to implementation, not to the theory baseline. The theory only requires that all real interval roots be found reliably enough for extrema detection.

14.15 Imposing A Velocity Limit In 1D

14.15.1 Problem Form

The constraint is:

$$|v(t)| \le v_{max} \qquad \forall t \in [t_0, t_f]$$

The question is not merely whether the current segment satisfies the limit, but how to modify the formulation so that it does.

14.15.2 Verification Step

The first step is always verification:

1. determine the polynomial coefficients;
2. compute max $|v(t)|$ over the interval using the extrema algorithm;
3. compare the result with v_{max}.

If:

$$\max |v(t)| \le v_{max}$$

then the constraint is already satisfied.

14.15.3 Constraint-Imposition Strategy

If the limit is violated, the standard project-level remedy is to modify the segment duration and recompute the coefficients.

The reason is structural:

- the endpoint conditions remain the same;
- the polynomial degree remains the same;
- the time law changes because the duration changes;
- the coefficients are then recomputed from the updated duration.

So the practical velocity-limit procedure is:

1. choose a candidate duration T;
2. determine the coefficients using `PolynomialCoefficientDetermination.md`;
3. compute max $|v(t)|$;
4. if the bound is violated, increase T and repeat;
5. stop when the limit is satisfied.

14.15.4 Duration Search

A robust algorithmic form is:

1. start from an initial duration $T^{(0)}$;
2. build a feasibility test:

$$F_v(T) = \max_{t\in[t_0,t_f]} |v(t;T)| - v_{max}$$

3. if $F_v(T) \leq 0$, the duration is feasible;
4. otherwise enlarge T and test again;
5. once a feasible interval is bracketed, optionally refine the smallest acceptable duration by bisection or another one-dimensional search method.

This turns velocity-limit imposition into a duration-search problem.

14.16 Imposing An Acceleration Limit In 1D

14.16.1 Problem Form

The constraint is:

$$|a(t)| \leq a_{max} \qquad \forall t \in [t_0, t_f]$$

14.16.2 Verification Step

1. determine the polynomial coefficients;
2. compute max $|a(t)|$ using the extrema algorithm driven by $j(t) = 0$;
3. compare the result with a_{max}.

14.16.3 Constraint-Imposition Strategy

If the bound is violated, the same logic applies:

- modify the duration;
- recompute the polynomial;
- recompute the acceleration extrema;
- repeat until the bound is satisfied.

Define:

$$F_a(T) = \max_{t\in[t_0,t_f]} |a(t;T)| - a_{max}$$

and search for a duration such that:

$$F_a(T) \leq 0$$

The method is the same as for velocity, but the monitored profile is acceleration instead of velocity.

14.17 Imposing A Jerk Limit In 1D

14.17.1 Problem Form

The constraint is:

$$|j(t)| \leq j_{max} \qquad \forall t \in [t_0, t_f]$$

14.17.2 Verification Step

1. determine the polynomial coefficients;
2. compute max $|j(t)|$ using the extrema algorithm driven by $s(t) = 0$;
3. compare the result with j_{max}.

14.17.3 Constraint-Imposition Strategy

If the bound is violated, define:

$$F_j(T) = \max_{t \in [t_0, t_f]} |j(t; T)| - j_{max}$$

and search for a duration satisfying:

$$F_j(T) \leq 0$$

Again, the general procedure is:

1. modify duration;
2. recompute coefficients;
3. recompute jerk extrema;
4. iterate until feasible.

14.18 A Caution About Time Scaling

The previous sections intentionally describe limit imposition through recomputation, not through a universal closed-form scaling formula.

That caution matters because, in the general project setting:

- physical endpoint derivative values may be prescribed;
- the coefficient solution depends on the chosen duration;
- changing T therefore changes the coefficient problem itself.

So, in the most general case treated here, it is safest to:

- recompute the polynomial for each tested duration;
- then re-evaluate the constrained profile.

14.18.1 Special Normalized-Profile Case

There is, however, an important special case.

If the normalized profile shape is already fixed and one only rescales time, then derivative magnitudes scale as:

$$v \sim \frac{1}{T}, \qquad a \sim \frac{1}{T^2}, \qquad j \sim \frac{1}{T^3}$$

This explains why increasing duration often helps reduce peak derivatives.

But in the baseline formulation of this project, the more general recompute-and-verify strategy is the safer one to adopt for requirements 8, 9, and 10.

14.19 Position, Velocity, Acceleration, Jerk, And Snap In Normalized Time

Using normalized time can simplify extrema analysis because every segment is studied on the same interval:

$$\tau \in [0, 1]$$

The candidate-point logic remains identical:

- extrema of $x(\tau)$ come from $\frac{dx}{d\tau} = 0$;
- extrema of $\frac{dx}{d\tau}$ come from $\frac{d^2x}{d\tau^2} = 0$;
- and so on.

The benefit is mainly organizational:

- one local interval for all segments;
- one uniform search domain;
- easier comparison across segments.

The cost is that physical derivative limits must still be interpreted through the scaling relations between t and τ.

14.20 What Changes In 3D

The scalar extrema logic does not disappear in 3D. It reappears in two distinct ways:

- component-wise analysis;
- vector-magnitude analysis.

14.20.1 Component-Wise Analysis

Each Cartesian component is treated as its own scalar polynomial:

$$x(t), \qquad y(t), \qquad z(t)$$

Then each component has its own profiles:

- $v_x(t), v_y(t), v_z(t)$;
- $a_x(t), a_y(t), a_z(t)$;
- $j_x(t), j_y(t), j_z(t)$;
- $p_x^{(4)}(t), p_y^{(4)}(t), p_z^{(4)}(t)$.

The scalar extrema algorithm can then be applied independently to each component.

14.20.2 Vector-Magnitude Analysis

Alternatively, one may analyse magnitudes such as:

$$\|\mathbf{v}(t)\|, \qquad \|\mathbf{a}(t)\|, \qquad \|\mathbf{j}(t)\|, \qquad \|\mathbf{p}^{(4)}(t)\|$$

This is often more physically meaningful when the concern is the total kinematic demand rather than the per-axis demand.

For example:

$$\|\mathbf{v}(t)\| = \sqrt{v_x(t)^2 + v_y(t)^2 + v_z(t)^2}$$

To find extrema of velocity magnitude, one can instead study:

$$\|\mathbf{v}(t)\|^2 = v_x(t)^2 + v_y(t)^2 + v_z(t)^2$$

because squaring preserves the location of maxima and minima for nonnegative magnitudes.

Its stationarity condition is:

$$\frac{d}{dt}\left(\|\mathbf{v}(t)\|^2\right) = 2\,\mathbf{v}(t) \cdot \mathbf{a}(t) = 0$$

Similarly:

- extrema of $\|\mathbf{a}(t)\|^2$ come from $\mathbf{a}(t) \cdot \mathbf{j}(t) = 0$;
- extrema of $\|\mathbf{j}(t)\|^2$ come from $\mathbf{j}(t) \cdot \mathbf{p}^{(4)}(t) = 0$.

14.21 Recommended 3D Interpretation For This Project

This document resolves the earlier open question deferred from `SymbolsAndNomenclature.md` and `NinthDegreePolynomialTheory.m`

The recommended project baseline is:

- use **component-wise analysis as the primary theory and implementation baseline**;
- use **vector-magnitude analysis as an additional derived check when the total 3D kinematic demand matters**.

This recommendation is consistent with the project assumptions because:

- the project is explicitly built from 1D to 3D;
- component-wise treatment is the direct extension of the scalar theory;
- it is simpler to derive, verify, and implement;
- it avoids introducing unnecessary coupling too early.

So, for the continuation of this project:

- the default interpretation of 3D constraints is component-wise;
- vector-magnitude analysis remains valid and useful, but secondary.

14.22 Relation To The Data Model

`02-data-model.md` defines:

$$\text{Constraints} = \{\text{constraintType}, \text{constraintValue}, \text{constantVelocitySelection}, \dots\}$$

From the viewpoint of this document:

- `constraintType` can represent velocity, acceleration, jerk, snap, time, or related segment-level restrictions;
- `constraintValue` is the bound associated with the selected type;
- `constraints[]` inside each segment is the place where one or more such restrictions are attached to that segment.

This document gives the theoretical meaning of those entries:

- analysis of maxima/minima tells whether a given profile violates a bound;
- duration search provides one baseline strategy to impose the bound;
- the same structure later supports the constant-velocity requirement addressed separately in `ConstantVelocitySegment`.

One implementation-oriented point remains open: if multiple limits must be imposed simultaneously on the same segment, the future software design may need a clearer representation than a single type/value pair viewed in isolation, even though the current `constraints[]` container already points in that direction.

14.23 Generic Constraint-Verification Algorithm

For a selected derivative order and a selected bound, the following procedure can be used.

1. Determine the segment coefficients.
2. Build the target profile:
   - $v(t)$ for velocity;
   - $a(t)$ for acceleration;
   - $j(t)$ for jerk.
3. Compute the candidate extrema points from the next derivative.
4. Evaluate the target profile at all candidates.
5. Compute the maximum absolute value.
6. Compare that result with the admissible limit.
7. Mark the segment as feasible or infeasible.

This algorithm is the verification core shared by requirements 8, 9, and 10.

14.24 Generic Constraint-Imposition Algorithm

The baseline limit-imposition strategy for the current project is:

1. choose the segment degree and boundary conditions;
2. choose an initial duration;
3. determine the polynomial coefficients;
4. verify the selected constraint;
5. if violated, increase duration and recompute;
6. repeat until the constraint is satisfied;
7. if desired, refine the smallest feasible duration.

This is conceptually simple, consistent with the current theory baseline, and directly compatible with the future desktop application.

14.25 Worked Symbolic Considerations

14.25.1 Example 1: Position Extrema

Suppose a degree-9 segment is written as:

$$x(t) = \sum_{i=0}^{9} a_i t^i$$

To determine the maximum and minimum position:

1. compute $v(t)$;
2. solve $v(t) = 0$ in $[t_0, t_f]$;
3. evaluate $x(t)$ at the interval endpoints and at the retained interior roots;
4. compare the values.

The result is exact at the level of the candidate-point principle, even if the interior roots themselves are obtained numerically.

14.25.2 Example 2: Velocity Limit

Suppose the segment must satisfy:

$$|v(t)| \leq v_{max}$$

The procedure is:

1. compute the coefficients for the current duration;
2. solve $a(t) = 0$ inside the interval;
3. evaluate $|v(t)|$ at all candidate points;
4. if the largest value exceeds v_{max}, enlarge duration and recompute.

This gives a clean theoretical basis for the future implementation of a velocity-limit tool.

14.25.3 Example 3: 3D Constraint Interpretation

Suppose a 3D trajectory has velocity vector:

$$\mathbf{v}(t) = \begin{bmatrix} v_x(t) \\ v_y(t) \\ v_z(t) \end{bmatrix}$$

The project baseline first checks:

$$|v_x(t)|, \qquad |v_y(t)|, \qquad |v_z(t)|$$

component by component.

If needed, a secondary check may also inspect:

$$\|\mathbf{v}(t)\|$$

to understand the total 3D kinematic demand.

14.26 Open Questions

- **Open — to resolve during software implementation** (`src/`): which numerical root solver should be adopted for interval root extraction of the stationarity polynomials?
- **Open — to resolve during software design**: when several limits are imposed simultaneously on the same segment, should the application search duration only, or also allow reformulations that change additional parameters?
- **Open — to resolve during later project phases**: should snap also receive an explicit limit-imposition requirement, beyond the extrema analysis already covered here?

14.27 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `NinthDegreePolynomialTheory.md`
- `BoundaryConditions.md`
- `PolynomialCoefficientDetermination.md`
- `ConstantVelocitySegment`
- `02-data-model.md`

15 Constant Velocity Segment

15.1 Intro And Scope

This document studies how to introduce a constant-velocity segment inside a trajectory originally described through a ninth-degree polynomial.

It covers Requirement 12 of `01-requirements.md`:

- a constant-speed segment is required inside a path originally defined as a degree-9 polynomial;
- the segment is specified by its start and end positions, either in absolute form or as percentages;
- the treatment starts in 1D and then extends to 3D.

This document builds on:

- `PolynomialCoefficientDetermination.md`, for segment recomputation from boundary data;
- `TrajectoryConstraints.md`, for the interpretation of velocity limits and duration search;
- `02-data-model.md`, for the `Constraints` fields that already reserve `constantVelocitySelection`, `constantVelocityValue`, and the start/end positions of the constant-velocity part.

The discussion remains symbolic-first and theoretical, consistent with `03-assumptions.md`.

15.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: duration of a generic segment;
- $\tau = \frac{t - t_0}{T}$: normalized time;
- $x(t)$: scalar position law in 1D;
- $\mathbf{p}(t)$: position vector in 3D;
- $v(t) = \dot{x}(t)$: scalar velocity in 1D;
- $\mathbf{v}(t) = \dot{\mathbf{p}}(t)$: velocity vector in 3D;
- v_{cv}: requested constant speed magnitude inside the constant-velocity segment;
- x_{cv}^{start}, x_{cv}^{end}: start and end positions of the constant-velocity subsegment in 1D;
- $\mathbf{p}_{cv}^{start}$, $\mathbf{p}_{cv}^{end}$: start and end positions of the constant-velocity subsegment in 3D;
- t_{cv}^{start}, t_{cv}^{end}: start and end times of the constant-velocity subsegment;
- $T_{cv} = t_{cv}^{end} - t_{cv}^{start}$: duration of the constant-velocity subsegment;
- $\sigma \in \{+1, -1\}$: 1D travel-direction sign;
- $\ell(t)$: arc-length function along the original 3D path, measured from the start of the selected interval;
- $\tau_{cv} \in [0, 1]$: local normalized time within the 3D constant-speed part (distinct from the direction sign σ above — see the notation note where it is introduced);
- $\ell_{cv}(\tau_{cv})$: arc length traveled within the constant-speed part, as a function of τ_{cv}.

When the constant-velocity segment is embedded between transition segments, the endpoint states of that middle segment are written as:

$$\mathcal{S}_{cv}^{start}, \qquad \mathcal{S}_{cv}^{end}$$

when compact notation is useful.

15.3 Path Vs Trajectory: Why This Topic Is Special

This requirement is easier to understand if the distinction from `04-future-topics.md` is made explicit:

- a **path** is the geometric locus to be followed;
- a **trajectory** is that path plus a time law.

A constant-velocity segment changes the time law in a very specific way:

- in 1D, it imposes constant signed velocity or constant speed magnitude over a selected interval;
- in 3D, it raises a choice between constant component-wise velocity and constant speed along the geometric path.

So the central question is not only “how do we keep velocity constant?”, but also:

- what exactly must remain unchanged from the original polynomial path;
- what is allowed to be reformulated;
- what continuity level is expected at the boundaries of the constant-velocity part.

15.4 First Key Fact: Exact Constant Velocity Is Very Restrictive

Suppose a single polynomial segment contains a nontrivial interval on which velocity is exactly constant.

In 1D, that means:

$$\dot{x}(t) = v_{cv}$$

for every t in some interval of nonzero length.

Equivalently:

$$\ddot{x}(t) = 0$$

throughout that interval.

But $\ddot{x}(t)$ is itself a polynomial. A polynomial that is zero on a nontrivial interval is zero everywhere. Therefore:

$$\ddot{x}(t) \equiv 0$$

for the whole segment, and the segment must be globally affine:

$$x(t) = c_0 + c_1 t$$

So, in general:

- an arbitrary ninth-degree polynomial cannot contain an exact constant-velocity subinterval while remaining the same single nontrivial ninth-degree polynomial;
- an exact constant-velocity part therefore requires a **reformulation** of the trajectory into multiple segments, unless the whole original segment is already linear.

This fact is foundational for the rest of the document.

15.5 1D Case: Exact Constant Velocity Segment

15.5.1 Geometric Interpretation

In 1D, the path is simply a line on the scalar axis. So “constant speed along the path” and “constant velocity magnitude” are almost the same notion, apart from sign.

If the motion direction over the selected interval is fixed, then the exact constant-velocity segment is:

$$x_{cv}(t) = x_{cv}^{start} + \sigma v_{cv}(t - t_{cv}^{start})$$

where:

- $\sigma \in \{+1, -1\}$ encodes the travel direction;
- $v_{cv} > 0$ is the requested speed magnitude.

The corresponding derivatives are:

$$\dot{x}_{cv}(t) = \sigma v_{cv}, \qquad \ddot{x}_{cv}(t) = 0, \qquad x_{cv}^{(3)}(t) = 0, \qquad x_{cv}^{(4)}(t) = 0$$

for the whole constant-velocity interval.

15.6 Duration Of The Constant-Velocity Part

Once the selected positions are known, the duration of the middle constant-velocity part is:

$$T_{cv} = \frac{|x_{cv}^{end} - x_{cv}^{start}|}{v_{cv}}$$

provided:

$$x_{cv}^{end} \neq x_{cv}^{start}, \qquad v_{cv} > 0$$

If the two positions coincide, the constant-velocity subsegment degenerates and the requirement loses meaning.

15.7 Why A Multi-Segment Reformulation Is Needed

If the original trajectory outside the selected interval is a degree-9 polynomial, then an exact constant-velocity part is normally introduced as a three-part structure:

1. entry transition segment;
2. constant-velocity middle segment;
3. exit transition segment.

In symbolic form:

$$\mathcal{T} = \{\text{entry transition}, \text{constant-velocity segment}, \text{exit transition}\}$$

The role of the two transition segments is to connect the surrounding motion to a middle segment whose higher derivatives are all zero.

15.8 Entry And Exit States In 1D

The middle segment has the natural endpoint states:

$$\mathcal{S}_{cv}^{start} = (x_{cv}^{start}, \sigma v_{cv}, 0, 0, 0)$$

$$\mathcal{S}_{cv}^{end} = (x_{cv}^{end}, \sigma v_{cv}, 0, 0, 0)$$

These states make the nature of the middle segment explicit:

- prescribed position at both ends;
- same velocity at both ends;
- zero acceleration, jerk, and snap at both ends.

The entry transition must end at $\mathcal{S}_{cv}^{start}$.

The exit transition must begin at $\mathcal{S}_{cv}^{end}$.

15.9 Baseline 1D Construction Strategy

The cleanest exact strategy in the current project is:

1. choose the two positions that delimit the constant-velocity interval;
2. define the target speed magnitude v_{cv};
3. compute the constant-velocity duration T_{cv};
4. create a middle affine segment with velocity σv_{cv};
5. construct an entry degree-9 transition that reaches $\mathcal{S}_{cv}^{start}$;
6. construct an exit degree-9 transition that leaves $\mathcal{S}_{cv}^{end}$ toward the later trajectory conditions;
7. determine the polynomial coefficients of the transition segments using `PolynomialCoefficientDetermination.md`.

This is the main exact 1D strategy recommended by this document.

15.10 How The Start And End Positions May Be Specified In 1D

The requirement allows the interval to be specified either in absolute form or by percentages.

15.10.1 Absolute Specification

The user gives:

$$x_{cv}^{start}, \qquad x_{cv}^{end}$$

directly.

This is the clearest formulation whenever the scalar coordinate range is already meaningful.

15.10.2 Percentage Specification

A simple percentage-based choice is:

$$\lambda_{start}, \lambda_{end} \in [0, 1]$$

with:

$$x_{cv}^{start} = x_0 + \lambda_{start}(x_f - x_0)$$

$$x_{cv}^{end} = x_0 + \lambda_{end}(x_f - x_0)$$

This is convenient, but it should be interpreted carefully:

- it is natural when the 1D motion is monotonic;
- it becomes less meaningful if the original polynomial overshoots or reverses direction.

So, in 1D, percentage specification is best treated as a convenience rule for monotonic path portions.

15.11 1D Consistency Checks

Before building the constant-velocity part, the following conditions should be checked.

1. The selected positions must lie on the intended path portion.
2. The ordering of the positions must be compatible with the chosen travel direction.
3. The requested speed must be positive:

$$v_{cv} > 0$$

4. The surrounding transition segments must have enough room to connect smoothly to zero higher derivatives at the boundaries of the constant-velocity segment.

The fourth point is especially important: a constant-velocity middle segment is easy to define, but smooth entry and exit may force the surrounding intervals to be long enough.

15.12 A Simpler But Less Smooth 1D Alternative

There is also a simpler formulation:

- keep only one polynomial segment before the constant part;
- insert the affine constant-velocity part;
- switch directly to the following part.

This is easy to define, but generally poor in smoothness because direct switching may produce discontinuities in acceleration, jerk, or snap.

So, unless low continuity is explicitly acceptable, the project baseline should prefer transition segments that match zero acceleration, jerk, and snap at the entry and exit of the constant-velocity part.

15.13 3D Case: Two Different Meanings Of "Constant Velocity"

In 3D, the topic becomes richer because two meanings must be separated.

15.13.1 Constant Velocity Vector

One may require:

$$\mathbf{v}(t) = \mathbf{v}_{cv}$$

constant over the selected interval.

Then the position law is:

$$\mathbf{p}_{cv}(t) = \mathbf{p}_{cv}^{start} + \mathbf{v}_{cv}(t - t_{cv}^{start})$$

This implies:

- straight-line motion along the chord joining the two selected points;
- constant speed magnitude;
- zero acceleration, jerk, and snap.

But this does **not** generally preserve the original curved polynomial path between the two selected positions.

15.13.2 Constant Speed Along The Original 3D Path

Alternatively, one may require that the motion follows the same geometric curve portion while travelling with constant speed magnitude:

$$\|\mathbf{v}(t)\| = v_{cv}$$

This preserves the path geometry, but the component velocities are no longer constant in general.

These two meanings coincide only in the special case where the selected path portion is already a straight line.

15.14 Decided 3D Interpretation For This Requirement

DECISION: for this requirement, the project adopts arc-length reparameterization (path-preserving, constant speed magnitude) as the baseline 3D interpretation, not just as a recommendation.

- preserve the selected geometric subpath of the original polynomial curve;
- impose constant **speed magnitude** along that preserved subpath.

Rationale:

- this interpretation offers the widest applicative range: it correctly handles a constant-speed requirement along a genuinely curved, multi-axis path, which the straight-chord alternative cannot do without deviating from the original path;
- in the practical case where the constant-speed requirement applies to motion along a single Cartesian axis — the most common real-world case for this kind of process constraint (e.g. avoiding acceleration-induced stress on the transported/machined body along one direction) — the original path in that interval is already a straight line, and the general arc-length construction reduces exactly (not approximately) to the same affine law used in the 1D case. No separate code path is needed for that case;
- the straight-chord alternative remains valid and simpler, but only coincides with path preservation when the selected subpath is already straight; for a genuinely curved multi-axis subpath it silently departs from the original geometry;
- the straight-chord-on-a-genuinely-curved-multi-axis-path case is deliberately not adopted as the project baseline now — it is deferred and will be added only if a concrete need for it arises (see Open Questions).

This decision was reached after clarifying two points: (a) only the straight-chord option keeps the velocity *vector* constant in both magnitude and direction — the arc-length option keeps only the magnitude constant, since direction varies continuously along a curve; (b) the two options coincide exactly, not just approximately, whenever the selected subpath is a straight line — which was verified algebraically by reducing the general arc-length construction to the 1D affine formula for that special case.

So, unlike the component-wise baseline adopted in `TrajectoryConstraints.md` for general constraint analysis, this document is primarily path-oriented:

- for general limits, component-wise treatment remains the default baseline;
- for this specific requirement, the decided baseline is constant speed along the selected path portion, via arc-length reparameterization.

15.15 3D Path-Preserving Strategy: Arc-Length Reparameterization

Let the original 3D polynomial path be:

$$\mathbf{p}_{orig}(t)$$

and let the selected subpath correspond to original parameter values:

$$t_s^{orig}, \qquad t_e^{orig}$$

with:

$$\mathbf{p}_{cv}^{start} = \mathbf{p}_{orig}(t_s^{orig}), \qquad \mathbf{p}_{cv}^{end} = \mathbf{p}_{orig}(t_e^{orig})$$

Define the arc-length function from the start of the selected interval:

$$\ell(t) = \int_{t_s^{orig}}^{t} \left\| \dot{\mathbf{p}}_{orig}(u) \right\| \, du$$

Then the total length of the selected subpath is:

$$L_{cv} = \ell(t_e^{orig})$$

If the requested constant speed magnitude is v_{cv}, the duration of the constant-speed traversal becomes:

$$T_{cv} = \frac{L_{cv}}{v_{cv}}$$

Now define a new local time variable over the constant-speed part:

$$\tau_{cv} = \frac{t - t_{cv}^{start}}{T_{cv}} \in [0, 1]$$

Notation note: this local normalized time is called τ_{cv}, not σ, even though σ was already used earlier in this document. That earlier $\sigma \in \{+1, -1\}$ is the 1D travel-direction sign (see *1D Case: Exact Constant Velocity Segment*) — an unrelated quantity that happens to share the same letter. Keeping both as σ would make this section read as if the direction sign were somehow becoming a normalized time variable. τ_{cv} also matches the project-wide convention that normalized time is written τ (see `SymbolsAndNomenclature`).

The corresponding traveled arc length is:

$$\ell_{cv}(\tau_{cv}) = \tau_{cv} L_{cv}$$

(named ℓ_{cv}, not s, to avoid colliding with s/**s**, which are reserved exclusively for snap).

The path-preserving constant-speed motion is then obtained by:

$$\mathbf{p}_{cv}(t) = \mathbf{p}_{orig}(\ell^{-1}(\ell_{cv}(\tau_{cv})))$$

This construction preserves the original curve and enforces constant speed magnitude along it.

15.16 Important Consequence In 3D

The arc-length-reparameterized constant-speed segment is generally **not polynomial in time anymore**.

This matters because it means:

- the original geometric path can be preserved;
- exact constant speed can be preserved;
- but the resulting time law is no longer, in general, a degree-9 polynomial in the new time variable.

So the 3D problem naturally separates into two layers:

- geometric path preservation;
- time-law reformulation.

This is not a defect. It is the mathematically natural outcome of the requirement.

15.16.1 Implementation Note: Where Numerical Methods Are Actually Needed

This non-polynomial consequence only bites in the genuinely curved, multi-axis case:

- **single-axis case** (the original subpath is already a straight line): as shown above, the arc-length construction reduces exactly to the closed-form affine law — evaluation stays polynomial, no numerical methods needed;
- **genuinely curved multi-axis case**: $\ell(t)$ generally has no closed form for a degree-9 polynomial path and must be computed via **numerical integration**; recovering $\ell^{-1}(\ell_{cv})$ then requires **numerical root-finding**. This is the only point in the theory library so far where evaluation departs from direct coefficient/Horner-based computation.

Implementation should branch on this distinction rather than always paying the numerical cost: detect the single-axis (straight-subpath) case and use the closed-form affine law directly, reserving numerical integration and root-finding for the genuinely curved multi-axis case.

15.17 3D Straight-Chord Alternative

If path preservation is not required, then the simpler exact construction is a straight-line constant-velocity segment:

$$\mathbf{p}_{cv}(t) = \mathbf{p}_{cv}^{start} + \frac{t - t_{cv}^{start}}{T_{cv}} \left(\mathbf{p}_{cv}^{end} - \mathbf{p}_{cv}^{start}\right)$$

with:

$$T_{cv} = \frac{\left\|\mathbf{p}_{cv}^{end} - \mathbf{p}_{cv}^{start}\right\|}{v_{cv}}$$

This gives:

- constant velocity vector;
- constant speed magnitude;
- straight motion between the selected points.

Its advantage is simplicity.

Its limitation is that the curved portion of the original polynomial path is replaced by its chord.

15.18 How The Start And End Positions May Be Specified In 3D

15.18.1 Absolute Specification

The user gives:

$$\mathbf{p}_{cv}^{start}, \qquad \mathbf{p}_{cv}^{end}$$

directly.

This is simple only if those points are already known on the original path.

15.18.2 Percentage Specification Along The Path

For path-oriented use, the most meaningful percentage convention is based on the path progression parameter over the selected original interval.

One convenient choice is to define:

$$\lambda_{start}, \lambda_{end} \in [0, 1]$$

on the original segment parameter interval:

$$t_s^{orig} = t_0 + \lambda_{start}(t_f - t_0)$$

$$t_e^{orig} = t_0 + \lambda_{end}(t_f - t_0)$$

and then:

$$\mathbf{p}_{cv}^{start} = \mathbf{p}_{orig}(t_s^{orig}), \qquad \mathbf{p}_{cv}^{end} = \mathbf{p}_{orig}(t_e^{orig})$$

This is parameter-percentage, not true arc-length percentage.

If a geometric percentage of traveled distance is preferred, then the percentage should be defined on arc length instead:

$$\ell(t_s^{orig}) = \lambda_{start} L_{tot}, \qquad \ell(t_e^{orig}) = \lambda_{end} L_{tot}$$

where L_{tot} is the total arc length of the original segment.

For this project, the cleanest recommendation is:

- in 1D, percentage by scalar position interval is acceptable on monotonic portions;
- in 3D, percentage by original parameter is the simplest practical convention;
- if precise geometric meaning is important in 3D, arc-length percentage is the more rigorous convention.

15.19 Transition Segments Around The Constant-Speed Part

Whether the constant-speed segment is 1D or 3D, smooth entry and exit normally require dedicated transition segments.

15.19.1 1D Transition Targets

In 1D, the target state at the start of the constant-speed interval is:

$$(x_{cv}^{start}, \sigma v_{cv}, 0, 0, 0)$$

and at the end:

$$(x_{cv}^{end}, \sigma v_{cv}, 0, 0, 0)$$

15.19.2 3D Transition Targets

For the straight-chord constant-velocity-vector option:

$$\mathbf{v}_{cv} = \frac{\mathbf{p}_{cv}^{end} - \mathbf{p}_{cv}^{start}}{T_{cv}}$$

and the transitions should match:

- position;
- velocity vector $\mathbf{v}_{cv}$;

- zero acceleration;
- zero jerk;
- zero snap.

For the arc-length-reparameterized path-preserving option, the velocity direction varies along the segment, so transition design is less trivial. In that case, the main requirement is smooth matching between:

- the preceding motion state;
- the reparameterized curve state at the beginning of the constant-speed interval;
- the corresponding state at the end.

This may lead to dedicated blending/reparameterization design choices in later documents.

15.20 Baseline Algorithm For Requirement 12

The following algorithm is the recommended baseline for the project.

1. Start from the original degree-9 trajectory segment.
2. Select the start and end of the desired constant-speed interval:
   - in absolute coordinates, or
   - through a chosen percentage convention.
3. Determine the corresponding positions:
   - x_{cv}^{start}, x_{cv}^{end} in 1D, or
   - $\mathbf{p}_{cv}^{start}$, $\mathbf{p}_{cv}^{end}$ in 3D.
4. Choose the requested constant speed magnitude v_{cv}.
5. Build the middle constant-speed segment:
   - affine in 1D;
   - arc-length-reparameterized in 3D if path preservation is required;
   - straight-chord affine in 3D if path preservation is not required.
6. Compute the duration T_{cv} of the middle segment.
7. Construct entry and exit transitions that connect smoothly to the middle segment.
8. Compute the transition coefficients with `PolynomialCoefficientDetermination.md`.
9. Verify continuity and duration consistency for the full three-part construction.

15.21 Relation To The Data Model

`02-data-model.md` already defines the following fields inside `Constraints`:

- `constantVelocitySelection`
- `constantVelocityValue`
- `constantVelocityInitialPos1D`
- `constantVelocityEndPos1D`
- `constantVelocityInitialPos`
- `constantVelocityEndPos`

This document provides the theoretical meaning of those fields:

- `constantVelocitySelection` activates the special subsegment logic;
- `constantVelocityValue` stores the requested speed magnitude v_{cv};
- the `1D` and `3D` position fields locate the start and end of the constant-speed part.

What the data model does **not** yet decide explicitly is:

- which percentage convention should be used when positions are not given absolutely;
- how transition segments are represented as part of the higher-level trajectory structure.

(Whether the 3D interpretation is path-preserving or straight-chord is no longer open — see *Decided 3D Interpretation For This Requirement* above.)

Those are theory-to-design bridges, not contradictions.

15.22 Worked Symbolic Considerations

15.22.1 Example 1: 1D Exact Constant-Velocity Middle Segment

Suppose the selected interval is:

$$x_{cv}^{start} \to x_{cv}^{end}$$

with requested speed magnitude v_{cv}.

Then the middle segment is:

$$x_{cv}(t) = x_{cv}^{start} + \sigma v_{cv}(t - t_{cv}^{start})$$

with duration:

$$T_{cv} = \frac{|x_{cv}^{end} - x_{cv}^{start}|}{v_{cv}}$$

The surrounding degree-9 transition segments must reach and leave this affine law with zero acceleration, jerk, and snap at the junctions.

15.22.2 Example 2: 3D Path-Preserving Constant-Speed Segment

Suppose the original polynomial path is:

$$\mathbf{p}_{orig}(t)$$

and two points on it are selected.

The geometric subpath between them is preserved, but the time law is replaced by an arc-length-based one:

$$\mathbf{p}_{cv}(t) = \mathbf{p}_{orig}(\ell^{-1}(\ell_{cv}(\tau_{cv})))$$

This preserves the original curve while enforcing constant speed magnitude.

15.22.3 Example 3: 3D Straight-Chord Alternative

If one instead chooses:

$$\mathbf{p}_{cv}(t) = \mathbf{p}_{cv}^{start} + \frac{t - t_{cv}^{start}}{T_{cv}} (\mathbf{p}_{cv}^{end} - \mathbf{p}_{cv}^{start})$$

then the motion is exactly constant-velocity in the vector sense, but the original curved subpath is replaced by a straight chord.

This makes the trade-off explicit:

- preserve path geometry, or
- preserve a simpler affine time law.

15.23 Open Questions

- **Open — to resolve if/when a concrete need arises**: the straight-chord alternative for a genuinely curved multi-axis subpath is deliberately not part of the current baseline (see *Decided 3D Interpretation*); should it be added later as an explicit user-facing option, and what would trigger that need?
- **Open — to resolve during data-format design**: when users specify the interval in percentages, which convention should be standardized in 3D: parameter percentage, arc-length percentage, or both?
- **Open — to resolve together with the later blending documents**: should the transition segments around the constant-speed part always enforce zero acceleration, jerk, and snap at the junctions, or should the continuity order be configurable?

- **Resolved**: the 3D arc-length section reused σ for the local normalized time, colliding with the 1D travel-direction sign σ defined earlier in this same document, and named the traveled-arc-length function $s(\cdot)$, colliding with s/**s** (reserved for snap). The local normalized time is now τ_{cv} and the traveled-arc-length function is now $\ell_{cv}(\cdot)$; see *3D Path-Preserving Strategy: Arc-Length Reparameterization*.

15.24 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `PolynomialCoefficientDetermination.md`
- `TrajectoryConstraints.md`
- `BlendingSegments`
- `TrajectoryPassingThroughConstraintPoints`
- `02-data-model.md`

16 Blending Segments

16.1 Intro And Scope

This document studies the smooth connection between consecutive trajectory segments when the final trajectory is **not** required to pass exactly through the nominal common endpoint of the original segments.

It covers Requirement 13 of `01-requirements.md`:

- complex trajectories are made of multiple segments;
- the connection between two consecutive segments is defined by:
  - the start of the connection inside the preceding segment;
  - the end of the connection inside the following segment;
- those two points may be specified in absolute form or by percentages;
- the treatment starts in 1D and then extends to 3D.

This document therefore addresses the classical "blend" situation:

- the original neighboring segments exist as nominal reference pieces;
- a local neighborhood around their nominal join is removed;
- a dedicated connecting segment is inserted instead;
- the final blended trajectory no longer needs to pass through the original common endpoint.

This topic is distinct from:

- `TrajectoryConstraints.md`, which studies limits inside a single segment;
- `ConstantVelocitySegment.md`, which studies a special subsegment with imposed constant speed;
- `TrajectoryPassingThroughConstraintPoints`, which will study the opposite case in which exact passage through the transition point must be preserved.

The discussion remains symbolic-first and theoretical, consistent with `03-assumptions.md`.

16.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- $x_k(t)$: preceding scalar segment in 1D;
- $x_{k+1}(t)$: following scalar segment in 1D;
- $\mathbf{p}_k(t)$: preceding vector segment in 3D;
- $\mathbf{p}_{k+1}(t)$: following vector segment in 3D;
- x_J: nominal common endpoint of two consecutive 1D segments before blending;
- $\mathbf{p}_J$: nominal common endpoint of two consecutive 3D segments before blending;
- $x_{b,k}^{start}$: point where blending starts inside segment k;
- $x_{b,k+1}^{end}$: point where blending ends inside segment $k+1$;
- $\mathbf{p}_{b,k}^{start}$: point where blending starts inside segment k in 3D;
- $\mathbf{p}_{b,k+1}^{end}$: point where blending ends inside segment $k+1$ in 3D;
- $t_{b,k}^{start}$: parameter value on segment k corresponding to the chosen blend-start point;

- $t_{b,k+1}^{end}$: parameter value on segment $k+1$ corresponding to the chosen blend-end point;
- T_b: duration of the blend segment;
- $\mathcal{S}_b^-$: state sampled from the preceding segment at the blend-start point;
- $\mathcal{S}_b^+$: state sampled from the following segment at the blend-end point.

When degree 9 is the main reference case, the two sampled 1D states are:

$$\mathcal{S}_b^- = (x, v, a, j, s)_b^- , \qquad \mathcal{S}_b^+ = (x, v, a, j, s)_b^+$$

where each component is obtained by evaluating the original neighboring segments and their derivatives at the selected blend points.

16.3 Why Blending Is Needed

A multi-segment trajectory defined only by nominal segment endpoints may suffer from one or more of the following issues at a join:

- a visible corner in the path;
- discontinuity in velocity;
- discontinuity in acceleration or higher derivatives;
- an overly abrupt change in kinematic demand.

Blending replaces the direct transition with a dedicated local connection that is smoother.

The key idea is:

- do not connect segment k directly to segment $k+1$ at the nominal join;
- truncate segment k before the join;
- truncate segment $k+1$ after the join;
- insert a new segment between the two truncation points.

So the final trajectory around the join becomes:

$$\{\text{truncated segment } k,\ \text{blend segment},\ \text{truncated segment } k+1\}$$

16.4 The Meaning Of "Without A Pass-Through Constraint"

Requirement 13 is fundamentally different from Requirement 14.

In the present document:

- the nominal join point is only a reference inherited from the original segmentation;
- the final blended trajectory is **not** forced to pass through that join;
- the true transition is spread over an interval.

This means that the original common endpoint loses its role as a mandatory interpolation point.

That relaxation is exactly what gives blending its geometric and kinematic flexibility.

16.5 Local Blend Window

Consider two nominal neighboring segments:

$$x_k(t), \qquad x_{k+1}(t)$$

in 1D, or:

$$\mathbf{p}_k(t), \qquad \mathbf{p}_{k+1}(t)$$

in 3D.

The blend is defined by two local points:

- one point inside the preceding segment;
- one point inside the following segment.

Those two points delimit the blend window.

In 1D:

$$x_{b,k}^{start}, \qquad x_{b,k+1}^{end}$$

In 3D:

$$\mathbf{p}_{b,k}^{start}, \qquad \mathbf{p}_{b,k+1}^{end}$$

The blend segment must connect the two sampled states attached to those points.

16.6 1D Blending Problem

16.6.1 Nominal Consecutive Segments

In 1D, let:

$$x_k(t), \qquad x_{k+1}(t)$$

be two consecutive nominal segments, with common nominal join value:

$$x_J$$

before blending.

Choose:

$$x_{b,k}^{start}$$

inside segment k, and:

$$x_{b,k+1}^{end}$$

inside segment $k+1$.

These two values identify the portion to be replaced.

16.7 Recovering The Corresponding Parameters

The selected blending points may be given as positions, but the original segments are parameterized by time or by a local normalized parameter. Therefore the first practical step is to recover:

$$t_{b,k}^{start}, \qquad t_{b,k+1}^{end}$$

such that:

$$x_k\left(t_{b,k}^{start}\right) = x_{b,k}^{start}$$

$$x_{k+1}\left(t_{b,k+1}^{end}\right) = x_{b,k+1}^{end}$$

If the original segments are monotonic over the relevant local portions, this recovery is conceptually simple.

If they are not monotonic, the same position value may correspond to multiple parameter values. In that case the intended branch must be selected explicitly.

This is one reason why percentage-based selection can be operationally simpler than absolute-position selection.

16.8 Sampling The Boundary States For The Blend

Once the two parameter values are known, the blending states are obtained directly from the original segments.

For the preceding segment:

$$\mathcal{S}_b^- = \left(x_k\left(t_{b,k}^{start}\right), \dot{x}_k\left(t_{b,k}^{start}\right), \ddot{x}_k\left(t_{b,k}^{start}\right), x_k^{(3)}\left(t_{b,k}^{start}\right), x_k^{(4)}\left(t_{b,k}^{start}\right)\right)$$

For the following segment:

$$\mathcal{S}_b^+ = \left(x_{k+1}\left(t_{b,k+1}^{end}\right), \dot{x}_{k+1}\left(t_{b,k+1}^{end}\right), \ddot{x}_{k+1}\left(t_{b,k+1}^{end}\right), x_{k+1}^{(3)}\left(t_{b,k+1}^{end}\right), x_{k+1}^{(4)}\left(t_{b,k+1}^{end}\right)\right)$$

for the degree-9 reference case.

These are not guessed or imposed independently. They are inherited from the neighboring segments.

16.9 Building The Blend Segment In 1D

The blend segment is a new local polynomial:

$$x_b(t)$$

with duration:

$$T_b$$

chosen for the blend interval.

In the degree-9 reference case, the blend segment is naturally determined by enforcing:

- position, velocity, acceleration, jerk, and snap at its start;
- position, velocity, acceleration, jerk, and snap at its end.

So the boundary data for the blend are exactly:

$$\mathcal{S}_b^-, \qquad \mathcal{S}_b^+$$

and the coefficient-determination problem is the same one already studied in `PolynomialCoefficientDetermination.md`.

This gives a very clean theoretical structure:

1. sample the two states from the original segments;
2. choose a blend duration T_b;
3. solve one standard degree-9 endpoint-interpolation problem.

16.10 Why Degree 9 Is A Natural Blend Reference

Degree 9 is especially convenient for blending because it can match:

- position;
- velocity;
- acceleration;
- jerk;
- snap;

at both ends of the blend.

This means that, in the reference case, the transition from the truncated original segment to the blend, and then from the blend to the following truncated segment, is smooth up to the fourth derivative.

That is a strong continuity level for a theoretical baseline.

16.11 Lower-Degree Analogues

The same blending logic applies to lower polynomial degrees, but with a reduced matched state.

Blend degree	Matched quantities at each end
3	x, v
5	x, v, a
7	x, v, a, j
9	x, v, a, j, s

So the general rule is:

- choose the blend degree;
- sample from the neighboring segments only the derivative orders that that degree can support at both ends;
- determine the blend coefficients accordingly.

16.12 Choosing The Blend Duration

The blend duration T_b is an actual design variable.

Several strategies are possible:

- inherit a duration from the removed original portions;
- specify T_b directly;
- search for T_b so that kinematic limits are respected.

The most important theoretical point is that changing T_b changes the blend coefficients.

So, exactly as in `TrajectoryConstraints.md`, duration and profile shape are coupled through coefficient recomputation.

16.13 Recommended 1D Baseline

The recommended baseline for this project is:

1. select the two blending points;
2. recover the corresponding local parameters on the neighboring segments;
3. sample the endpoint states from those segments;
4. choose a blend duration;
5. compute the blend coefficients in normalized time;
6. verify the resulting blend against any active constraints;
7. if needed, adjust T_b and recompute.

This keeps the whole procedure aligned with the rest of the theory library.

16.14 Absolute Vs Percentage Specification In 1D

Requirement 13 allows the blend points to be given either in absolute form or by percentages.

16.14.1 Absolute Specification

The user gives:

$$x_{b,k}^{start}, \qquad x_{b,k+1}^{end}$$

directly.

This is intuitive when the scalar positions themselves are meaningful and the corresponding branches on the original segments are unambiguous.

16.14.2 Percentage Specification

A convenient local parameter-based form is:

$$\lambda_k^{start}, \qquad \lambda_{k+1}^{end} \in [0,1]$$

with:

$$\tau_{b,k}^{start} = \lambda_k^{start}$$

$$\tau_{b,k+1}^{end} = \lambda_{k+1}^{end}$$

where each τ is the local normalized coordinate of its own segment.

This is often operationally cleaner than absolute-position selection because:

- no inverse position lookup is needed;
- the choice is unambiguous even when a segment is not monotonic in position.

For this reason, local percentage selection is a particularly natural baseline for blending.

16.15 1D Blend Does Not Need To Pass Through The Nominal Join

Once the blend segment is built, the final trajectory around the connection is:

$$\{x_k \text{ up to } t_{b,k}^{start},\ x_b,\ x_{k+1} \text{ from } t_{b,k+1}^{end}\}$$

The nominal common point between the original segments is no longer enforced.

This is exactly the behavior intended in Requirement 13.

16.16 3D Blending Problem

16.16.1 Geometric Meaning In 3D

In 3D, blending is first a geometric operation and only then a coefficient-determination problem.

Let:

$$\mathbf{p}_k(t), \qquad \mathbf{p}_{k+1}(t)$$

be two nominal consecutive 3D segments.

Choose:

$$\mathbf{p}_{b,k}^{start}, \qquad \mathbf{p}_{b,k+1}^{end}$$

as the two vector points delimiting the blend window.

These are points on the original neighboring trajectories, not arbitrary free-space points detached from them.

16.17 Recovering The Corresponding Parameters In 3D

The practical goal is again to recover:

$$t_{b,k}^{start}, \qquad t_{b,k+1}^{end}$$

such that:

$$\mathbf{p}_k\left(t_{b,k}^{start}\right) = \mathbf{p}_{b,k}^{start}$$

$$\mathbf{p}_{k+1}\left(t_{b,k+1}^{end}\right) = \mathbf{p}_{b,k+1}^{end}$$

If the blend points are specified through local percentages, these parameters are obtained directly.

If they are specified as absolute 3D points, one must identify the corresponding parameter values on the original segments. This is straightforward only when the intended point-to-parameter correspondence is unique.

So, just as in 1D, percentage-based selection is often cleaner operationally.

16.18 Sampling The Boundary States In 3D

Once the two parameter values are known, the vector states are sampled from the original segments.

For the preceding segment:

$$\mathcal{S}_b^- = \left(\mathbf{p}_k, \dot{\mathbf{p}}_k, \ddot{\mathbf{p}}_k, \mathbf{p}_k^{(3)}, \mathbf{p}_k^{(4)}\right)_{t=t_{b,k}^{start}}$$

For the following segment:

$$\mathcal{S}_b^+ = \left(\mathbf{p}_{k+1}, \dot{\mathbf{p}}_{k+1}, \ddot{\mathbf{p}}_{k+1}, \mathbf{p}_{k+1}^{(3)}, \mathbf{p}_{k+1}^{(4)}\right)_{t=t_{b,k+1}^{end}}$$

in the degree-9 reference case.

These vector states define the blend boundary conditions.

16.19 How The 3D Blend Is Actually Computed

Even though the blend is defined geometrically in 3D, the coefficient calculation is still most naturally performed component-wise.

That is:

- one scalar blend is computed for the x component;
- one scalar blend is computed for the y component;
- one scalar blend is computed for the z component;
- all three share the same blend duration T_b.

So the recommended interpretation is:

- **geometric selection** of the blend window in 3D;
- **component-wise coefficient determination** once the endpoint vector states are known.

This is consistent with the overall project architecture:

- geometry is handled at the vector level;
- polynomial determination is reused from the scalar theory.

16.20 Why The 3D Blend Is Still Different From Purely Independent Axis Work

Although the actual coefficient determination is component-wise, the 3D blending problem is not just "three unrelated 1D blends".

The coupling comes from:

- the shared geometric meaning of the chosen blend points;
- the shared blend duration;
- the fact that the same truncation event affects all axes simultaneously.

So the correct project-level picture is:

- geometric synchronization in 3D;
- scalar coefficient solving underneath.

16.21 Straight-Line Shortcut Vs Genuine 3D Blend

A tempting simplification in 3D would be to connect:

$$\mathbf{p}_{b,k}^{start} \quad \text{to} \quad \mathbf{p}_{b,k+1}^{end}$$

with a straight line.

That is not the general blending strategy adopted here.

The reason is that Requirement 13 is about smooth connection between neighboring segments, not about replacing the local transition with an arbitrary chord disconnected from the inherited kinematic state.

Therefore the baseline strategy is:

- sample the full states from the original segments;
- determine a dedicated blend segment consistent with those states.

This preserves the local motion information carried by the nominal segments.

16.22 Absolute Vs Percentage Specification In 3D

16.22.1 Absolute Specification

The user gives:

$$\mathbf{p}_{b,k}^{start}, \qquad \mathbf{p}_{b,k+1}^{end}$$

directly.

This is meaningful when the intended points on the original segments are already known unambiguously.

16.22.2 Percentage Specification

A particularly practical choice is local parameter percentage:

$$\lambda_k^{start}, \qquad \lambda_{k+1}^{end} \in [0,1]$$

applied to the normalized parameter of each segment.

Then:

$$\tau_{b,k}^{start} = \lambda_k^{start}, \qquad \tau_{b,k+1}^{end} = \lambda_{k+1}^{end}$$

and the corresponding points are obtained by direct evaluation of the original segments.

This avoids point-to-parameter inversion and is therefore a strong practical baseline for 3D as well.

16.23 Relation Between Blending And Constraints

A blend is primarily a continuity device, but it may also have to satisfy active constraints.

Once a blend segment has been determined, one may still need to verify:

- velocity limits;
- acceleration limits;
- jerk limits;
- other project-specific restrictions.

So blending and constraints are not separate universes:

- blending decides how the local connection is shaped;
- constraints decide whether that shape is admissible.

If the blend violates active limits, one baseline remedy is to modify the blend duration T_b and recompute.

16.24 Baseline Blending Algorithm

The following procedure is the recommended baseline for Requirement 13.

1. Start from two nominal consecutive segments.
2. Select the blend-start point inside the preceding segment.
3. Select the blend-end point inside the following segment.
4. Recover the corresponding local parameters.
5. Evaluate the neighboring segments and their derivatives at those parameters.
6. Form the two endpoint states of the blend.
7. Choose a blend degree and a blend duration.
8. Determine the blend coefficients using `PolynomialCoefficientDetermination.md`.
9. Replace the removed local portions of the original segments with the new blend segment.
10. Verify the resulting blend against active constraints and adjust duration if needed.

This procedure applies directly in 1D and extends to 3D by vector-state sampling plus component-wise coefficient determination.

16.25 Relation To The Data Model

`02-data-model.md` defines:

Blend =

- `nextSegmentBlendingPos1D`,
- `previousSegmentBlendingPos1D`,
- `nextSegmentBlendingPos`,
- `previousSegmentBlendingPos`

This document gives the theoretical meaning of those fields:

- `nextSegmentBlendingPos1D` identifies where blending toward the next segment starts in 1D;
- `previousSegmentBlendingPos1D` identifies where blending from the previous segment ends in 1D;
- `nextSegmentBlendingPos` identifies where blending toward the next segment starts in 3D;
- `previousSegmentBlendingPos` identifies where blending from the previous segment ends in 3D.

At a higher level, `Trajectory1D` and `Trajectory3D` also contain:

segmentEndpointRespect

whose meaning becomes especially relevant here.

For Requirement 13, the natural interpretation is:

- if exact respect of nominal segment endpoints is required, then ordinary blending is not the right mechanism;
- if exact endpoint passage is **not** required, then the blend settings become active and meaningful.

So this document is the theoretical counterpart of the branch:

- `segmentEndpointRespect = no` for ordinary blending;
- `segmentEndpointRespect = yes` for the later exact-pass-through topic of Requirement 14.

16.26 Worked Symbolic Considerations

16.26.1 Example 1: 1D Blend Around A Nominal Join

Suppose two nominal scalar segments meet at a nominal point x_J.

Choose:

$$x_{b,k}^{start}$$

before the nominal join, and:

$$x_{b,k+1}^{end}$$

after the nominal join.

The blend segment is then computed from the states sampled at those two points, not from the nominal join itself.

So the final trajectory is smoother, but it no longer has to interpolate x_J.

16.26.2 Example 2: Degree-9 Blend

In the degree-9 case, the sampled start and end states are:

$$(x, v, a, j, s)_b^- \quad \text{and} \quad (x, v, a, j, s)_b^+$$

These ten scalar values are exactly the right number to determine one degree-9 blend segment uniquely once T_b is fixed.

16.26.3 Example 3: 3D Blend

Suppose two 3D segments are truncated at:

$$\mathbf{p}_{b,k}^{start}, \qquad \mathbf{p}_{b,k+1}^{end}$$

The blend boundary data are the two vector states sampled there.

The blend is then computed by solving:

- one scalar polynomial problem for x;
- one scalar polynomial problem for y;
- one scalar polynomial problem for z;

with one shared duration T_b.

This is the cleanest way to combine 3D geometry with scalar polynomial theory.

16.27 Open Questions

- **Open — to resolve during later software design**: should the blend duration T_b be user-specified, inherited automatically from the removed original portions, or optimized under active constraints?
- **Open — to resolve during data-format design**: should blend points be stored primarily as absolute points, as local percentages, or as both representations?
- **Open — to resolve together with `TrajectoryPassingThroughConstraintPoints`**: should the application expose Requirement 13 and Requirement 14 as two clearly separated modes, or as one continuity tool with a mandatory pass-through toggle?

16.28 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `PolynomialCoefficientDetermination.md`
- `TrajectoryConstraints.md`
- `ConstantVelocitySegment.md`
- `TrajectoryPassingThroughConstraintPoints`
- `02-data-model.md`

17 Trajectory Passing Through Constraint Points

17.1 Intro And Scope

This document studies the multi-segment trajectory case in which the path must pass **exactly** through the specified segment endpoints while still remaining smooth across the transitions.

It covers Requirement 14 of `01-requirements.md`:

- the ordinary smooth-connection approach of Requirement 13 is not sufficient when the path must pass exactly through the segment endpoints;
- the transition points must coincide with the specified segment endpoints;
- the trajectory must avoid a stop-and-go behavior at those interior points;
- velocity, acceleration, jerk, and snap are assumed to be zero at the start of the first segment and at the end of the last segment;
- the treatment starts in 1D and then extends to 3D.

This document is therefore the natural counterpart of `BlendingSegments.md`:

- `BlendingSegments.md` studies the case where the nominal join may be bypassed;
- the present document studies the case where the nominal join is itself a mandatory interpolation point.

The discussion remains symbolic-first and theoretical, consistent with `03-assumptions.md`.

17.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- $x_k(t)$: the k-th scalar trajectory segment in 1D;
- $\mathbf{p}_k(t)$: the k-th vector trajectory segment in 3D;
- x_i: the i-th required waypoint in 1D;
- $\mathbf{p}_i$: the i-th required waypoint in 3D;
- $\mathcal{S}_i$: state attached to the i-th waypoint;
- v_i, a_i, j_i, s_i: velocity, acceleration, jerk, and snap assigned to waypoint i in 1D;
- $\mathbf{v}_i, \mathbf{a}_i, \mathbf{j}_i, \mathbf{p}_i^{(4)}$: waypoint derivatives in 3D;
- T_k: duration of segment k;
- $\tau_k \in [0, 1]$: local normalized time of segment k;
- N: number of segments;
- $M = N + 1$: number of exact waypoints.

For the degree-9 reference case, the state attached to waypoint i in 1D is:

$$\mathcal{S}_i = (x_i, v_i, a_i, j_i, s_i)$$

and, in 3D:

$$\mathcal{S}_i = \left(\mathbf{p}_i, \mathbf{v}_i, \mathbf{a}_i, \mathbf{j}_i, \mathbf{p}_i^{(4)}\right)$$

17.3 Why Requirement 13 Is Not Enough

The ordinary blending strategy of Requirement 13 removes a neighborhood around the nominal join and replaces it with a dedicated connection segment.

That is useful when:

- exact passage through the nominal join is not required;
- geometric and kinematic smoothing are allowed to shift the effective transition away from the original endpoint.

Requirement 14 changes the problem completely.

Here:

- the specified segment endpoints are mandatory interpolation points;
- the final trajectory must pass exactly through them;
- yet the motion should not collapse into a zero-velocity stop at every interior point.

So the key challenge is:

- exact waypoint passage;
- smooth multi-segment continuity;
- nontrivial motion through the interior waypoints.

This is no longer a local blend-replacement problem. It becomes a **global piecewise trajectory construction** problem.

17.4 Exact Pass-Through In 1D

17.4.1 Waypoint Sequence

Suppose the desired scalar trajectory must pass through the ordered waypoints:

$$x_0,\ x_1,\ x_2,\ \dots,\ x_N$$

with one segment between each consecutive pair.

Then segment k connects:

$$x_k \to x_{k+1}$$

for:

$$k = 0, 1, \dots, N-1$$

The final trajectory must satisfy:

$$x_k(t_k^{end}) = x_{k+1}$$

and:

$$x_{k+1}(t_{k+1}^{start}) = x_{k+1}$$

at the shared interior waypoint.

So the waypoint is not bypassed. It is interpolated exactly by both neighboring segments.

17.4.2 Why A Stop-And-Go Solution Is Undesirable

An easy but poor solution would be:

- make each segment begin and end with zero velocity;
- do the same for acceleration, jerk, and possibly snap.

That guarantees exact passage through every waypoint, but it also creates a stop event at each one.

In symbolic form, this undesirable interior choice would be:

$$v_i = 0$$

at every interior waypoint i.

Requirement 14 explicitly aims to avoid this behavior.

So the project needs a formulation in which:

- the trajectory passes exactly through every waypoint;
- the shared interior state is smooth;
- interior velocity is generally nonzero unless the geometry or timing truly forces it to vanish.

17.4.3 Boundary Conditions At The Global Ends

The requirement fixes the boundary behavior at the beginning and at the end of the whole trajectory:

$$v_0 = a_0 = j_0 = s_0 = 0$$

$$v_N = a_N = j_N = s_N = 0$$

for the degree-9 reference case.

These are global start/end conditions, not conditions to be imposed at every interior waypoint.

17.4.4 Interior Waypoint States

At each interior waypoint:

$$x_i, \qquad i = 1, \dots, N-1$$

the position is known because the trajectory must pass through that point exactly.

The derivatives:

$$v_i, \qquad a_i, \qquad j_i, \qquad s_i$$

are not, in general, known in advance.

They must be treated as:

- either design variables specified by the user or by a higher-level rule;
- or unknown internal variables determined by a global solution strategy.

This is the central conceptual difference from ordinary endpoint interpolation of a single segment.

17.5 Segment Representation In 1D

17.5.1 One Segment Per Consecutive Pair

For the degree-9 reference case, segment k can be written as:

$$x_k(\tau_k), \qquad \tau_k \in [0, 1]$$

with duration:

$$T_k$$

and boundary states:

$$\mathcal{S}_k^{start} = (x_k, v_k, a_k, j_k, s_k)$$

$$\mathcal{S}_k^{end} = (x_{k+1}, v_{k+1}, a_{k+1}, j_{k+1}, s_{k+1})$$

Once those two states and the duration T_k are known, the coefficients of segment k are determined exactly by the standard degree-9 coefficient procedure from `PolynomialCoefficientDetermination.md`.

17.5.2 Automatic Continuity Through Shared States

If consecutive segments are built with the same interior waypoint state, continuity is automatic.

For example, at waypoint x_i:

$$\mathcal{S}_i^{shared} = (x_i, v_i, a_i, j_i, s_i)$$

If segment $i - 1$ ends with that state and segment i starts with that same state, then the full trajectory is continuous up to snap at that waypoint.

So the continuity requirement does not need an extra correction step. It is built into the shared-state formulation itself.

17.6 Why The Full Problem Is Global

17.6.1 Local Segment Solves Are Easy

Once the waypoint states and segment durations are known, each segment is easy:

- it is just a standard endpoint interpolation problem;
- it can be solved independently from the others.

17.6.2 Determining The Interior States Is The Hard Part

The actual difficulty is to determine:

$$v_i, \quad a_i, \quad j_i, \quad s_i$$

for every interior waypoint.

If only the positions:

$$x_0,\ x_1,\ \ldots,\ x_N$$

and the endpoint zero-derivative conditions are prescribed, then the problem is not uniquely determined yet.

This is because:

- the segment coefficients depend on the waypoint states;

- the waypoint states are shared across segments;
- the exact passage constraints alone do not uniquely fix all those interior derivatives.

Therefore Requirement 14 requires a **global rule** in addition to exact interpolation.

17.7 Three Families Of Global Strategy

The interior waypoint states may be determined in three broad ways.

17.7.1 Prescribed Interior Derivatives

One may decide explicitly:

$$v_i, \quad a_i, \quad j_i, \quad s_i$$

at each interior waypoint.

This is the most direct strategy, but it requires the user or a higher-level planner to know those values.

17.7.2 Heuristic Interior Derivatives

One may compute interior derivatives from neighboring waypoint positions and durations by a deterministic rule, for example:

- finite-difference-inspired velocity estimates;
- smoothed slope estimates;
- recursively derived higher derivatives.

This is simpler than full optimization, but it is still a design choice rather than a mathematically unique consequence of the interpolation constraints.

17.7.3 Optimization-Based Global Determination

One may treat the interior waypoint derivatives as unknowns and determine them by minimizing a global smoothness cost while preserving exact waypoint passage.

Typical examples are:

- minimum-jerk-type criteria;
- minimum-snap-type criteria;
- weighted combinations of derivative costs.

This is the most systematic strategy and is the cleanest theoretical baseline for the present requirement.

17.8 Recommended 1D Baseline For This Project

The recommended baseline is:

- exact pass-through at every waypoint;
- zero velocity, acceleration, jerk, and snap only at the global start and end;
- interior waypoint derivatives treated as unknown shared state variables;
- those unknowns determined by a global smoothness criterion;
- each segment then reconstructed from the resulting shared states.

For the degree-9 reference case, a natural smoothness criterion is to minimize a functional related to snap, because:

- degree 9 naturally matches snap at both ends;
- snap already has a privileged role in the project's degree-9 theory;
- this criterion strongly penalizes overly abrupt changes in the motion law.

So the practical baseline picture is:

1. choose the exact waypoints;
2. choose the segment durations;
3. treat interior derivative values as global unknowns;
4. impose the global start/end derivative zeros;

5. solve for the interior states through a smoothness criterion;
6. reconstruct every segment using standard coefficient determination.

17.9 Exact Pass-Through Versus Blending

The difference from Requirement 13 can now be stated sharply.

In ordinary blending:

- the original join point may disappear from the final trajectory;
- the transition is distributed over a local blend window.

In exact pass-through mode:

- every specified waypoint remains a mandatory interpolation point;
- no local bypass is allowed;
- continuity is achieved by shared states at the waypoint itself.

So the two methods are not small variations of the same thing. They are genuinely different trajectory-construction modes.

17.10 Segment Durations In 1D

The durations:

$$T_0,\ T_1,\ \dots,\ T_{N-1}$$

remain important design parameters.

As in the previous documents:

- changing a duration changes the segment coefficients;
- changing durations also changes the globally optimal or heuristic interior derivatives;
- duration selection and exact pass-through smoothness are therefore coupled.

Possible strategies are:

- user-specified durations;
- durations derived from nominal geometric spacing;
- durations refined later to satisfy active constraints.

This keeps Requirement 14 naturally connected to `TrajectoryConstraints.md`.

17.11 Lower-Degree Versions

The same exact-pass-through logic applies to degrees lower than 9, but with a reduced shared state.

Degree	Shared waypoint state in the default symmetric formulation
3	x_i, v_i
5	x_i, v_i, a_i
7	x_i, v_i, a_i, j_i
9	x_i, v_i, a_i, j_i, s_i

So Requirement 14 is not conceptually specific to degree 9, but degree 9 is the richest reference case because it supports shared continuity up to snap.

17.12 3D Exact Pass-Through

17.12.1 Exact Waypoints In 3D

Now suppose the trajectory must pass exactly through:

$$\mathbf{p}_0,\ \mathbf{p}_1,\ \mathbf{p}_2,\ \cdots,\ \mathbf{p}_N$$

with one segment between consecutive vector waypoints.

The same conceptual structure remains:

- every waypoint is mandatory;
- interior stop events are not desired;
- continuity is enforced by shared waypoint states.

17.12.2 Shared Waypoint States In 3D

For the degree-9 reference case, waypoint i carries:

$$\mathcal{S}_i = \left(\mathbf{p}_i, \mathbf{v}_i, \mathbf{a}_i, \mathbf{j}_i, \mathbf{p}_i^{(4)}\right)$$

The global end conditions become:

$$\mathbf{v}_0 = \mathbf{a}_0 = \mathbf{j}_0 = \mathbf{p}_0^{(4)} = \mathbf{0}$$

$$\mathbf{v}_N = \mathbf{a}_N = \mathbf{j}_N = \mathbf{p}_N^{(4)} = \mathbf{0}$$

and the interior waypoint derivatives are shared unknown vectors.

17.12.3 How The 3D Segments Are Computed

As in the previous documents, the 3D coefficient computation is still most naturally carried out component-wise.

That is:

- solve one scalar pass-through problem for the x axis;
- solve one scalar pass-through problem for the y axis;
- solve one scalar pass-through problem for the z axis;
- keep the segment durations synchronized across the three axes.

So the correct project-level interpretation is:

- exact waypoint geometry is handled at the vector level;
- coefficient determination is reused from the 1D theory, component by component.

17.12.4 Avoiding Zero Velocity In 3D

The stop-and-go issue in 3D is not only:

$$\mathbf{v}_i = \mathbf{0}$$

at an interior waypoint, but also the broader case in which the shared interior state is chosen so poorly that the trajectory effectively stalls or changes direction unnaturally.

So the same principle applies:

- do not force interior waypoint derivatives to zero unless the geometry or timing genuinely requires it;
- determine them globally so that exact passage and smooth through-motion coexist.

17.13 Recommended 3D Baseline

The recommended baseline in 3D is:

- exact interpolation of all specified waypoints;
- zero derivative vectors only at the global start and end;
- shared interior vector states across neighboring segments;
- component-wise segment reconstruction once those states are known;
- one synchronized set of segment durations across all axes.

This is fully consistent with the overall project philosophy:

- start from the 1D theory;
- extend to 3D through structured vector composition.

17.14 Why Requirement 14 Is More Constrained Than Requirement 13

Requirement 13 gives geometric freedom because the transition may avoid the nominal endpoint.

Requirement 14 removes that freedom because:

- the waypoint must be hit exactly;
- the same point belongs to both neighboring segments;
- smoothness must be achieved with no geometric bypass.

So Requirement 14 is intrinsically more constrained.

This is why a local blend insertion is no longer sufficient and a global waypoint-state strategy becomes necessary.

17.15 Baseline Algorithm

The following procedure is the recommended baseline for Requirement 14.

1. Define the exact waypoint sequence.
2. Choose the polynomial degree for the segments.
3. Choose or initialize the segment durations.
4. Impose zero velocity, acceleration, jerk, and snap at the global start and end for the degree-9 reference case.
5. Introduce shared unknown derivative states at the interior waypoints.
6. Determine those interior states by a global rule:
   - direct prescription,
   - heuristic estimation,
   - or, preferably, a smoothness-based global solve.
7. Reconstruct each segment from its start and end waypoint states using `PolynomialCoefficientDetermination.md`.
8. Verify continuity and exact waypoint interpolation.
9. Verify active kinematic constraints and, if needed, update durations and recompute.

This algorithm applies directly in 1D and extends to 3D by vector waypoint states and component-wise segment reconstruction.

17.16 Relation To The Data Model

`02-data-model.md` currently contains:

$$\text{Trajectory1D}\{\text{Segment1D}[nn],\ \text{segmentEndpointRespect}\}$$

$$\text{Trajectory3D}\{\text{Segment3D}[nn],\ \text{segmentEndpointRespect}\}$$

This document gives a precise theoretical interpretation to:

$$\text{segmentEndpointRespect}$$

For Requirement 14, the natural interpretation is:

- `segmentEndpointRespect = yes`;
- exact passage through the specified segment endpoints is mandatory;
- ordinary blending settings are therefore not the primary mechanism.

This complements the interpretation already given in `BlendingSegments.md`:

- `segmentEndpointRespect = no` activates ordinary blending logic;
- `segmentEndpointRespect = yes` activates exact-pass-through logic.

At the state level, the current data model does not yet explicitly store the shared interior waypoint derivatives needed by this document. So one future refinement will likely need a more explicit representation of:

- waypoint state variables;
- segment durations;
- global continuity mode.

17.17 Worked Symbolic Considerations

17.17.1 Example 1: Three Waypoints In 1D

Suppose the trajectory must pass exactly through:

$$x_0,\ x_1,\ x_2$$

with two segments.

The global endpoint conditions are:

$$v_0 = a_0 = j_0 = s_0 = 0$$

$$v_2 = a_2 = j_2 = s_2 = 0$$

The interior waypoint state is:

$$\mathcal{S}_1 = (x_1, v_1, a_1, j_1, s_1)$$

If:

$$v_1 \neq 0$$

then the trajectory can pass through x_1 without stopping there.

17.17.2 Example 2: Difference From Ordinary Blending

In `BlendingSegments.md`, two neighboring segments may be truncated before and after their nominal join, and the final blended curve may avoid the original join completely.

Here, that is not allowed.

The two neighboring segments must both interpolate the exact shared waypoint:

$$x_i$$

or:

$$\mathbf{p}_i$$

depending on dimension.

So the geometric freedom is smaller, but the waypoint fidelity is exact.

17.17.3 Example 3: 3D Exact Pass-Through

Suppose the 3D trajectory must interpolate:

$$\mathbf{p}_0,\ \mathbf{p}_1,\ \mathbf{p}_2$$

with zero derivative vectors at the global start and end.

Then the interior shared state is:

$$\left(\mathbf{p}_1, \mathbf{v}_1, \mathbf{a}_1, \mathbf{j}_1, \mathbf{p}_1^{(4)}\right)$$

Once that interior state and the segment durations are fixed, both segments are determined by standard endpoint interpolation applied component-wise.

17.18 Open Questions

- **Open — to resolve during later software design**: should the project adopt one explicit global smoothness criterion as the default exact-pass-through rule, such as a minimum-snap cost, or should multiple selectable criteria be exposed?
- **Open — to resolve together with duration handling**: should segment durations in exact-pass-through mode be specified directly by the user, derived automatically from waypoint spacing, or refined through a joint duration-and-state search?
- **Open — to resolve during data-structure design**: should shared interior waypoint derivatives be represented explicitly as trajectory-level state objects, or should they remain implicit variables reconstructed only during computation?

17.19 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `BoundaryConditions.md`
- `PolynomialCoefficientDetermination.md`
- `TrajectoryConstraints.md`
- `BlendingSegments.md`
- `02-data-model.md`